INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

INFORME DEL PROYECTO: FUNCIONALIDADES, PRUEBAS Y DISEÑO MÓVIL

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Descripción del proyecto 3
1.1 Objetivos 3
1.2 Módulos 3
1.3 Tecnologías 3
1.4 Cómo se conectan las partes 4
2. Endpoints definidos 6
2.1 Formato único de respuesta 7
3. Registro y login diferenciados 7
3.1 Fragmentos de código 8
3.2 Pantallas de registro 9
4. Qué puede hacer cada rol 10
4.1 Recepcionista 10
4.2 Almacenero 16
4.3 Administrador 18
4.4 Programador 23
4.5 Cliente 26
5. Pantallas de error 28
6. Registro de incidentes 30
7. Pruebas de rendimiento y estrés con k6 31
7.1 Prueba de humo 32
7.2 Límite de intentos de login 33
7.3 Escalones de carga (5, 10, 25 y 50 usuarios) 34
7.4 Conclusión de las pruebas 37
8. Diseño de la aplicación Android e iOS 37
8.1 Tecnología 40
8.2 Seguridad y cambios en la API 40
8.3 Plan de construcción 40
9. Conclusiones 41

1. Descripción del proyecto

LeGrandHotel es un sistema web para gestionar un hotel. Reúne en un solo lugar el trabajo diario del personal (reservas, check-in y check-out, habitaciones, huéspedes, cochera e inventario) y la reserva en línea de los huéspedes, que eligen el tipo de habitación, ven lo que incluye y el total, y el hotel les asigna una habitación libre.

El sistema tiene dos partes: una API REST que guarda los datos y aplica todas las reglas del negocio, y una página web (aplicación de una sola página) que la consume. La misma API servirá también a la aplicación móvil para Android e iOS que se diseña en la sección 8.

1.1 Objetivos

- Evitar errores de operación: que una habitación no se reserve dos veces en las mismas fechas, que no haya dos huéspedes en la misma habitación y que el inventario no quede en negativo.
- Que cada persona vea solo lo que le corresponde según su rol, y que la API lo haga cumplir aunque alguien intente entrar escribiendo la dirección a mano.
- Dejar rastro: cada cambio queda en la auditoría con quién lo hizo y cuándo, y cada error tiene un código de incidente para encontrarlo en el registro.
- Medir el rendimiento con pruebas de carga antes de publicar el sistema.

1.2 Módulos

Módulo	Qué cubre
Reservas y estadías	Reservas con cálculo del total en el servidor, check-in (crea la estadía y ocupa la habitación) y check-out (la habitación pasa a limpieza)
Habitaciones y tipos	Habitaciones por piso y estado; tipos con precio por noche, capacidad y lo que incluyen
Huéspedes	Fichas con documento, teléfono y nacionalidad; se vinculan con la cuenta web si el huésped tiene una
Cochera	Espacios, entrada y salida de vehículos, con o sin estadía asociada
Inventario	Insumos con stock y mínimo; entradas y salidas con motivo
Personal y cuentas	Fichas del personal, registro de cuentas y cambio de roles
Reportes y auditoría	Ocupación, ingresos de los últimos 6 meses y quién cambió qué
Portal del cliente	Entrada con Google, datos del huésped, reserva en línea y sus reservas
Vista del programador	Salud de la API, latencias, peticiones, errores y rutas, sin entrar al servidor

1.3 Tecnologías

Parte	Tecnología	Para qué se usa
API (backend)	PHP 8.2 y Laravel 12	Rutas REST, validación, reglas del negocio y acceso a datos
Autenticación	Laravel Sanctum	Tokens Bearer con vencimiento (60 minutos) para la web y la app móvil
Inicio con Google	Laravel Socialite (OAuth 2.0)	Cuentas de clientes sin contraseña
Base de datos	MySQL 8.4	Datos del hotel con claves foráneas, índices y transacciones

Parte	Tecnología	Para qué se usa
Página web (frontend)	React 18, TypeScript y Vite	Interfaz de una sola página, rápida y con tipos
Datos en la web	TanStack Query y Axios	Caché de consultas, paginación y manejo único de errores 401/403/429/5xx
Navegación	React Router	Rutas protegidas por rol y pantallas de error
Pruebas de la API	Pest (PHPUnit)	121 pruebas automáticas de reglas, permisos y errores
Pruebas de la web	Vitest y Testing Library	25 pruebas de rutas, formularios y permisos
Calidad de código	PHPStan nivel 5, Laravel Pint, ESLint y TypeScript estricto	Detectar errores antes de ejecutar y mantener un estilo único
Pruebas de carga	k6 (Grafana)	Medir tiempos de respuesta y errores con muchos usuarios a la vez
Entorno local	Docker Compose	La misma base MySQL para todo el equipo
App móvil (diseño)	React Native con Expo y TypeScript	Una sola base de código para Android e iOS
Control de versiones	Git y GitHub	Historial y trabajo en equipo

Nota: el servidor donde se publicará el sistema todavía no está definido. Todo lo de este informe se ejecutó en el entorno local del equipo.

1.4 Cómo se conectan las partes

La página web y la app móvil no guardan datos: le piden todo a la API en formato JSON con el token de la sesión. La API valida cada petición (FormRequest), revisa el rol (middleware role:) y las reglas por registro (policies), y responde siempre con el mismo formato, sea éxito o error. La base de datos MySQL tiene 16 tablas del sistema relacionadas por 13 claves foráneas; el diagrama de la página siguiente se generó leyendo el esquema real de la base.

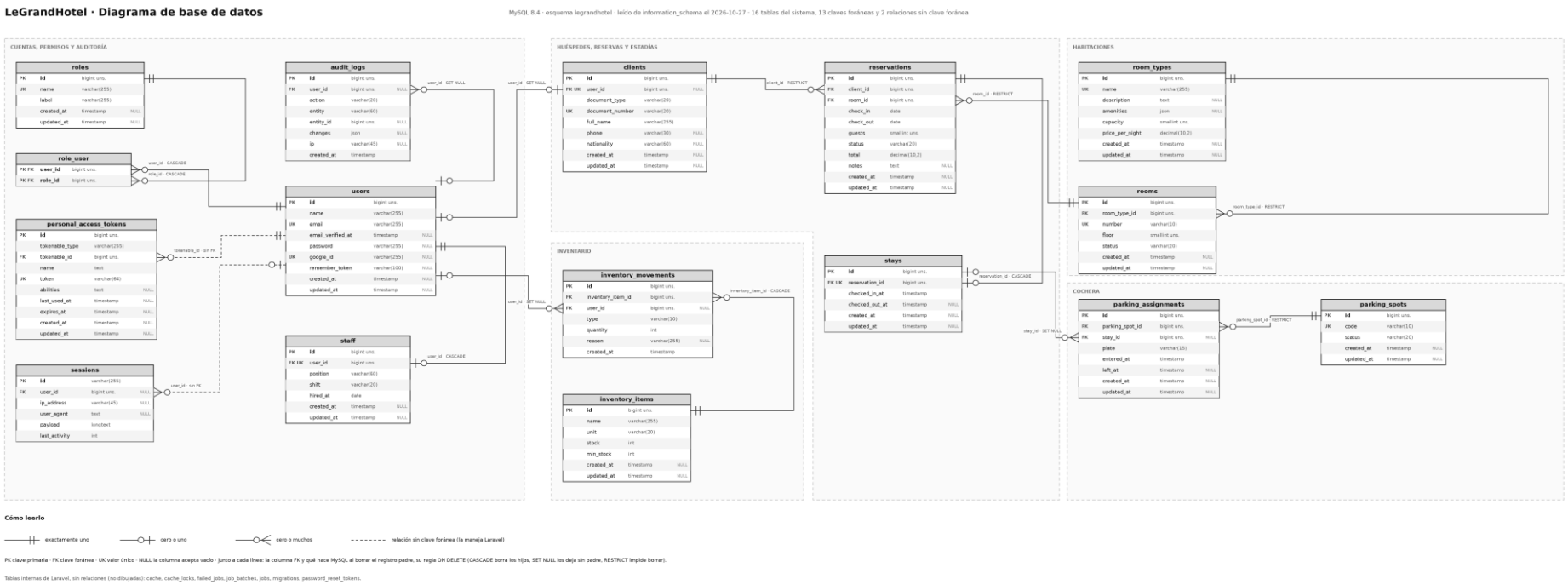


Figura 1. Diagrama de la base de datos (MySQL), generado desde information_schema: tablas, columnas, claves y regla ON DELETE de cada relación. Versión ampliable en docs/diagramas/bd/diagrama-bd.pdf.

2. Endpoints definidos

Todas las rutas viven bajo /api/v1 y responden JSON. Son 77 rutas (contadas con php artisan route:list). En la tabla, «CRUD» significa listar (GET), crear (POST), ver (GET /{id}), editar (PUT/PATCH) y eliminar (DELETE).

Grupo	Método y ruta	Quién puede usarla
Salud	`GET /health`	Público
Autenticación	`POST /auth/login`	Público (5 intentos por minuto)
	`GET /auth/google/redirect · /auth/google/callback`	Público (clientes)
	`GET /auth/me · POST /auth/logout · POST /auth/refresh`	Cualquier sesión
	`POST /auth/register`	Administrador y Programador (3 por minuto)
Correo	`GET /email/verify/{id}/{hash} · POST /email/verification-notification`	Enlace firmado · sesión
Portal del cliente	`PUT /portal/profile`	Cliente
	`GET /portal/room-types`	Cliente
	`GET · POST /portal/reservations`	Cliente
	`POST /portal/reservations/{id}/cancel`	Cliente (solo las suyas)
Panel	`GET /dashboard`	Administrador, Recepcionista, Almacenero, Programador
Tipos de habitación	`GET /room-types · /room-types/{id}`	Administrador y Recepcionista
	`POST · PUT · DELETE /room-types`	Administrador
Reservas	`CRUD /reservations · POST /reservations/{id}/check-in`	Administrador y Recepcionista
Estadías	`GET /stays · /stays/{id} · POST /stays/{id}/check-out`	Administrador y Recepcionista
Habitaciones	`CRUD /rooms · GET /rooms/available · PATCH /rooms/{id}/status`	Administrador y Recepcionista
Huéspedes	`CRUD /clients`	Administrador y Recepcionista
Cochera	`CRUD /parking-spots · CRUD /parking-assignments`	Administrador y Recepcionista
Inventario	`CRUD /inventory-items · GET · POST /inventory-items/{id}/movements`	Administrador y Almacenero
Personal	`CRUD /staff`	Administrador
Usuarios	`CRUD /users · PUT /users/{id}/roles`	Administrador
Reportes	`GET /reports/occupancy · /reports/income`	Administrador
Auditoría	`GET /audit-logs`	Administrador
Programador	`GET /dev/overview · /dev/requests · /dev/errors · /dev/routes`	Programador y Administrador

2.1 Formato único de respuesta

Todas las respuestas usan App\Support\ApiResponse. Un éxito trae success y data (y meta en los listados paginados, con un máximo de 50 por página); un error trae un código estable que la web usa para decidir qué mostrar, nunca HTML ni trazas del servidor.

Respuesta de error (ejemplo real: doble reserva)

```
{
  "success": false,
  "error": {
    "code": "ROOM_ALREADY_BOOKED",
    "message": "La habitación ya está reservada en esas fechas."
  }
}
```

Código HTTP	Cuándo responde la API	Qué muestra la web
401	Sin sesión o token vencido	Pantalla «Tu sesión terminó» y vuelve al login
403	El rol no tiene permiso	Pantalla «No tienes acceso a esta sección»
404	El registro o la ruta no existe	Pantalla «No encontramos esta página»
409	Regla del negocio: doble reserva, habitación ocupada, stock insuficiente, Programador único…	Mensaje en el formulario
422	Datos inválidos (FormRequest)	Error junto a cada campo
429	Demasiados intentos (Retry-After)	Pantalla con cuenta regresiva
500	Error inesperado, con código de incidente	Pantalla 500 con el código para soporte
503	Servicio no disponible o sin conexión	Pantalla que reintenta sola

3. Registro y login diferenciados

Hay dos tipos de cuenta con caminos separados, tanto en la pantalla como en la API. Las reglas se validan en el servidor: aunque alguien modifique la página, la API rechaza lo que no corresponde.

	Personal del hotel	Cliente (huésped)
Roles	Administrador, Recepcionista, Almacenero, Programador	Cliente
Cómo entra	Correo y contraseña (POST /auth/login)	Solo con Google (OAuth); no tiene contraseña
Quién crea la cuenta	El Administrador o el Programador, desde «Registrar personal». No hay registro público	Se crea sola la primera vez que entra con Google
Datos extra	Ficha de personal: cargo, turno y fecha de ingreso	DNI (8 dígitos) y teléfono (9 dígitos), una sola vez
Protección	5 intentos de login por minuto; contraseña cifrada con bcrypt; mínimo 8 caracteres con letras y números	Google verifica el correo; un DNI no puede quedar ligado a dos cuentas
A dónde llega	Al panel de su rol	A «Completar registro» y luego a «Reservar»

El Programador es único. Ni el Administrador ni el Programador pueden crear otra cuenta de Programador, darle ese rol a otra persona, quitárselo al que existe o eliminarlo. La API responde 409 DEVELOPER_UNIQUE o DEVELOPER_LOCKED si se intenta.

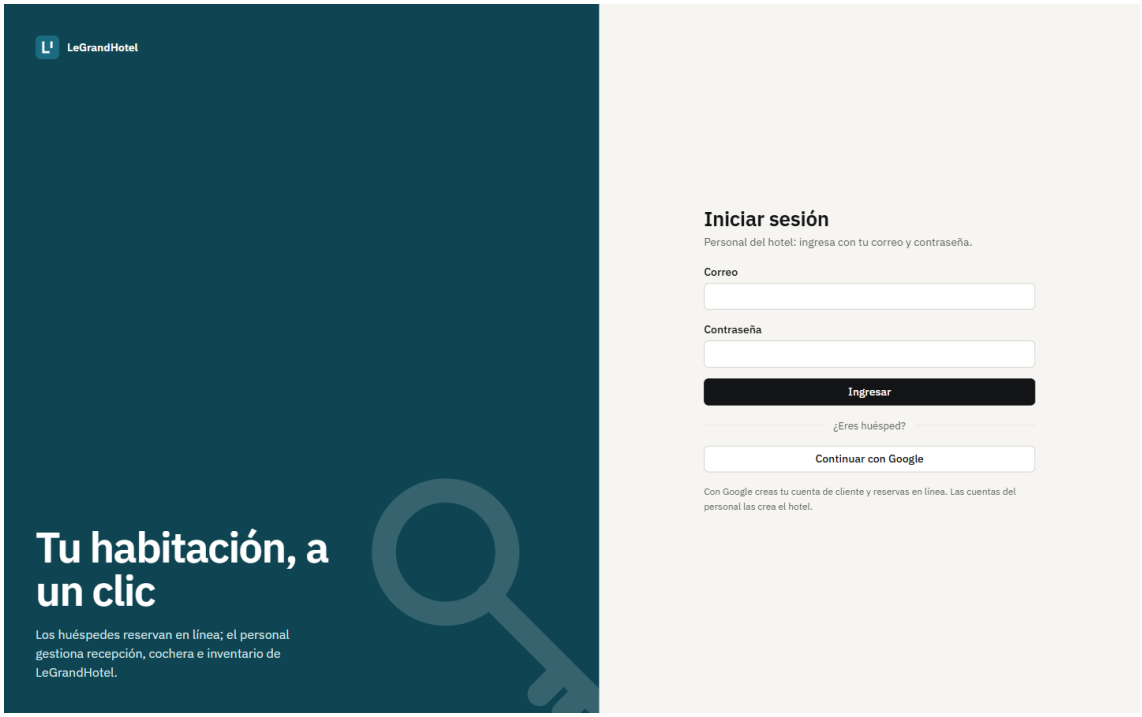


Figura 2. Login: el personal entra con correo y contraseña; el huésped, con «Continuar con Google».

3.1 Fragmentos de código

El login con contraseña solo deja entrar al personal; un cliente que intente usarlo recibe 403:

api/app/Http/Controllers/Api/AuthController.php

```
if (! $user || ! $user->password || ! Hash::check($request->string('password'), $user-
>password)) {
    throw new ApiException('INVALID_CREDENTIALS', 'Correo o contraseña incorrectos.',
401);
}
// Con contraseña solo entra el personal; los clientes entran con Google (no tienen
contraseña).
if (! $user->hasRole(...Role::INTERNAL)) {
    throw new ApiException('ACCOUNT_WITHOUT_ACCESS', 'Esta cuenta no tiene acceso al
sistema del hotel.', 403);
}
```

El registro del personal solo acepta los roles permitidos, y está protegido en la ruta por rol y por límite de intentos:

api/routes/api.php y api/app/Http/Requests/Auth/RegisterRequest.php

```
Route::post('register', [AuthController::class, 'register'])
    ->middleware(['role:admin,developer', 'throttle:register']);

'password' => ['required', 'confirmed', Password::min(8)->letters()->numbers()],
'role'     => ['required', Rule::in(Role::ASSIGNABLE)],   // Recepción, Almacén o
Administración
```

Google es solo para clientes: si el correo ya pertenece a alguien del personal, se le devuelve al login con un aviso:

api/app/Http/Controllers/Api/GoogleAuthController.php

```
$user = User::where('google_id', $googleUser->getId())->orWhere('email', $googleUser-
>getEmail())->first();
// Google es solo para clientes: el personal entra con su correo y contraseña.
if ($user && ! $user->hasRole(Role::CLIENT)) {
    return redirect()->away("{$frontend}/login?error=google_personal");
}
```

En la web, cada ruta declara qué roles la pueden abrir; si no corresponde, se muestra la pantalla 403 (y la API también responde 403):

web/src/auth/ProtectedRoute.tsx

```
export function ProtectedRoute({ roles }: { roles?: RoleName[] }) {
  const { user, hasRole } = useAuth()
  if (!user) return <Navigate to="/login" replace />
  if (roles && !hasRole(...roles)) return <ForbiddenPage />
  return <Outlet />
}
```

3.2 Pantallas de registro

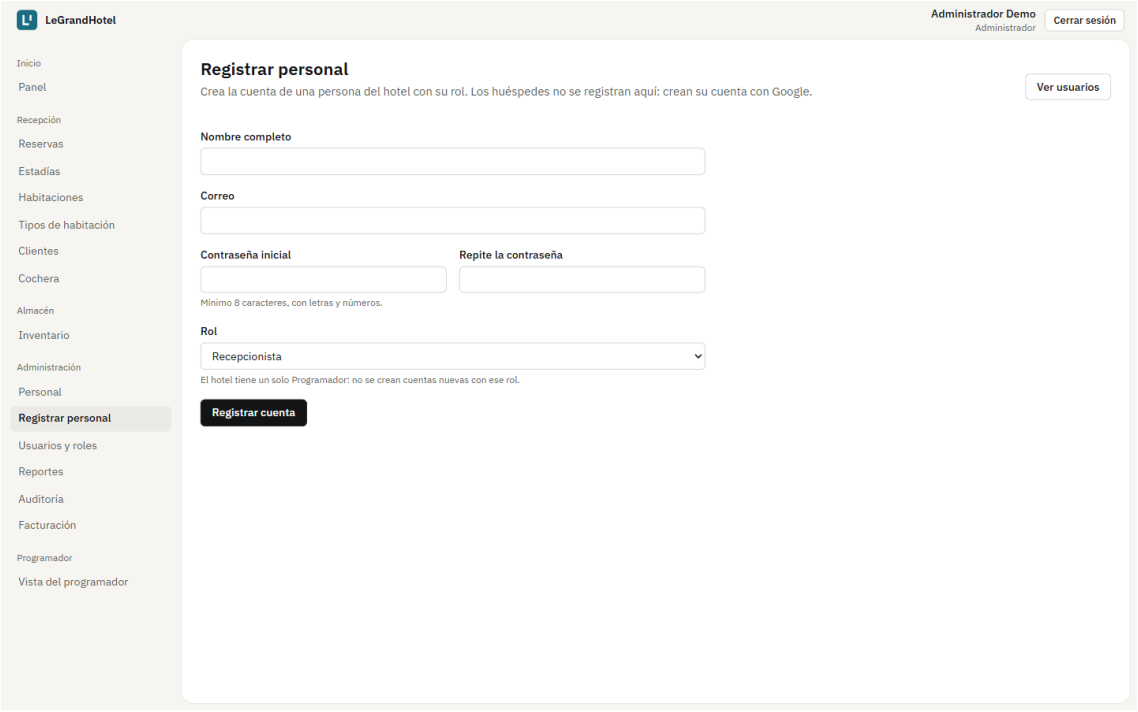


Figura 3. Registrar personal (Administrador).

Crea la cuenta de una persona del hotel con su contraseña inicial. Solo ofrece Recepcionista, Almacenero y Administrador; el aviso recuerda que el hotel tiene un solo Programador. La nueva cuenta recibe un correo para verificarse.

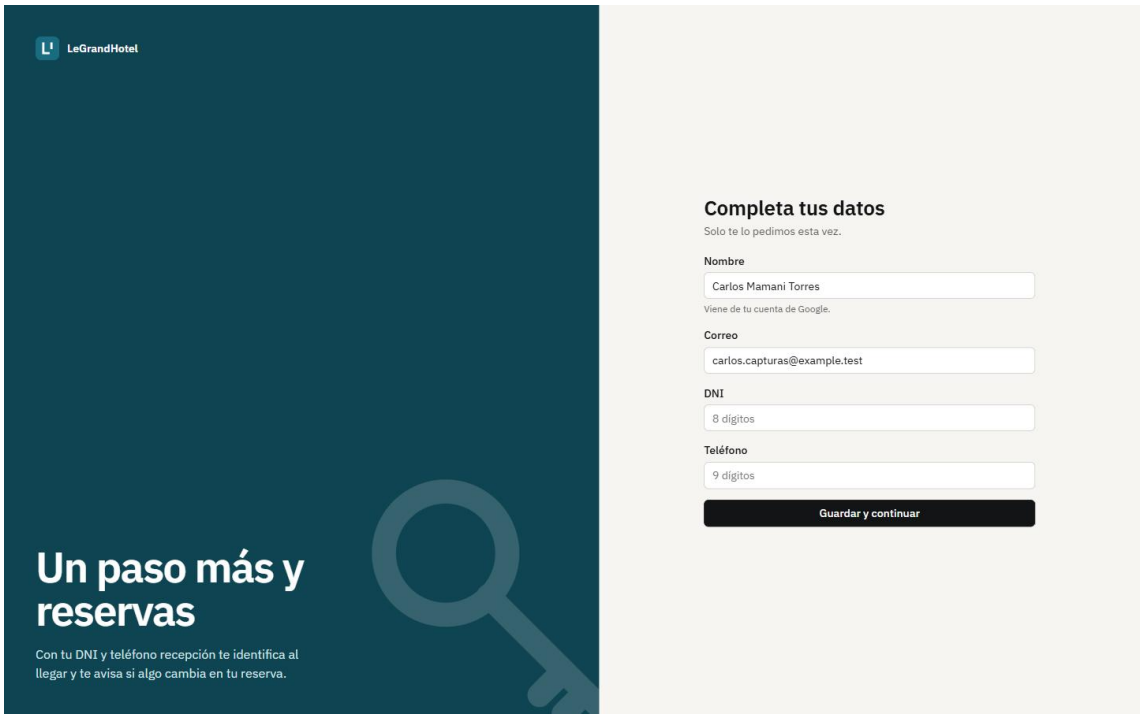


Figura 4. Completar registro del cliente después de Google.

El nombre y el correo vienen de Google y no se editan. El huésped escribe su DNI y su teléfono una sola vez; si su DNI ya estaba registrado en recepción, se vincula esa ficha en vez de crear otra.

4. Qué puede hacer cada rol

Cada rol ve en el menú solo sus pestañas. Las capturas de esta sección se tomaron el 27 de octubre de 2026 con la página funcionando sobre la base MySQL del proyecto, con los datos de demostración (70 habitaciones, 1000 huéspedes y 5000 reservas).

Pestaña	Admin.	Recep.	Almacén	Program.	Cliente
Panel	✓	✓	✓	—	—
Reservas, Estadías, Habitaciones, Clientes, Cochera	✓	✓	—	—	—
Tipos de habitación	edita	ve	—	—	—
Inventario	✓	—	✓	—	—
Personal, Usuarios y roles, Reportes, Auditoría, Facturación	✓	—	—	—	—
Registrar personal	✓	—	—	✓	—
Vista del programador	✓	—	—	✓	—
Completar registro, Reservar, Mis reservas	—	—	—	—	✓

4.1 Recepcionista

Atiende a los huéspedes en el día a día: reserva por ellos, hace el check-in y el check-out, controla la cochera y el estado de las habitaciones.

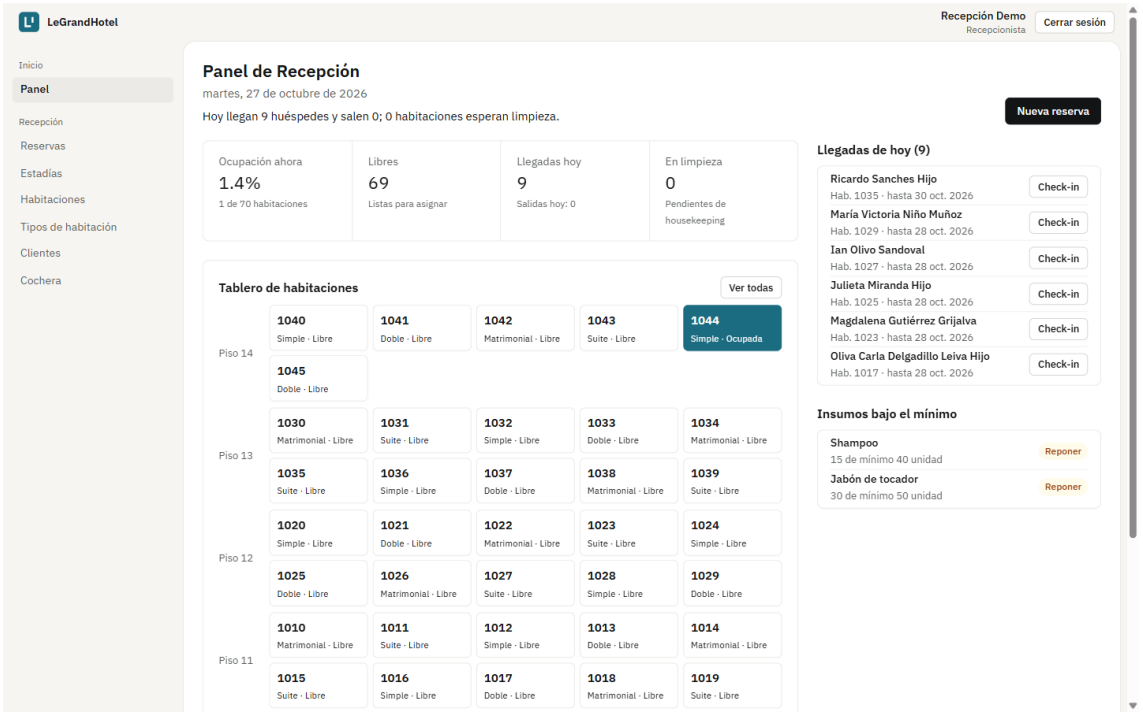


Figura 5. Panel de Recepción.

Resumen del día: ocupación, llegadas y salidas de hoy con su botón de check-in, ingresos del mes y el tablero de habitaciones por piso con su estado (libre, ocupada, limpieza o mantenimiento).

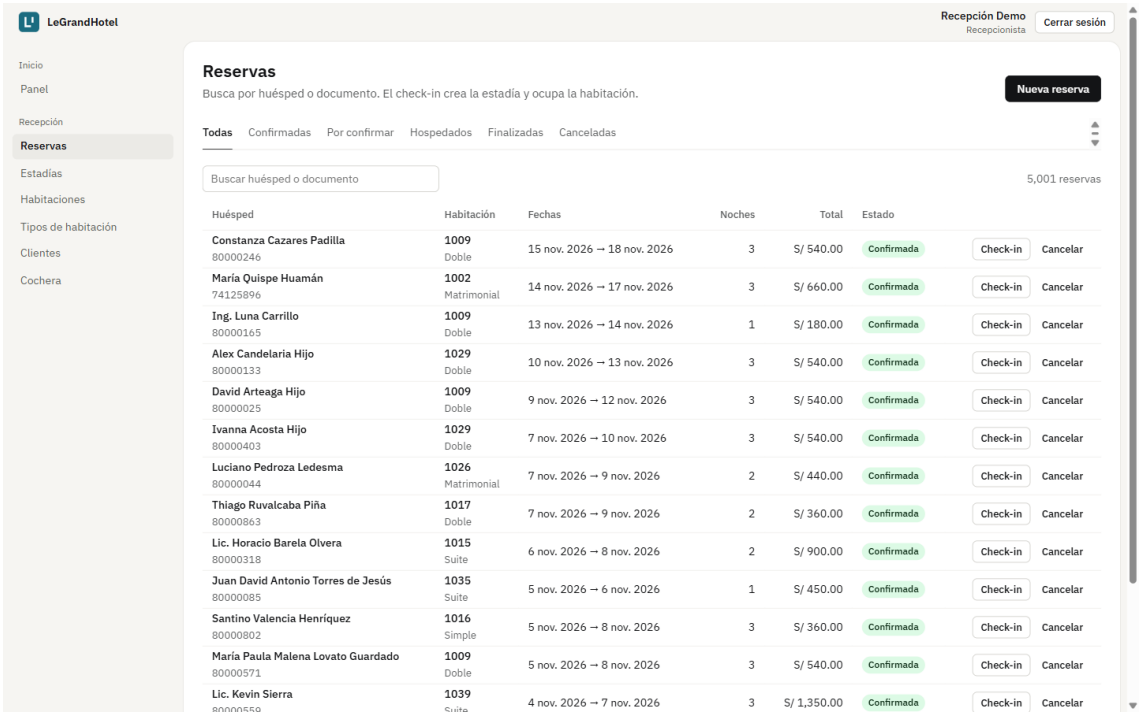


Figura 6. Reservas.

Lista paginada en el servidor con filtros por estado (confirmadas, hospedados, finalizadas, canceladas) y búsqueda por huésped o documento. Desde aquí se hace el check-in o se cancela una reserva.

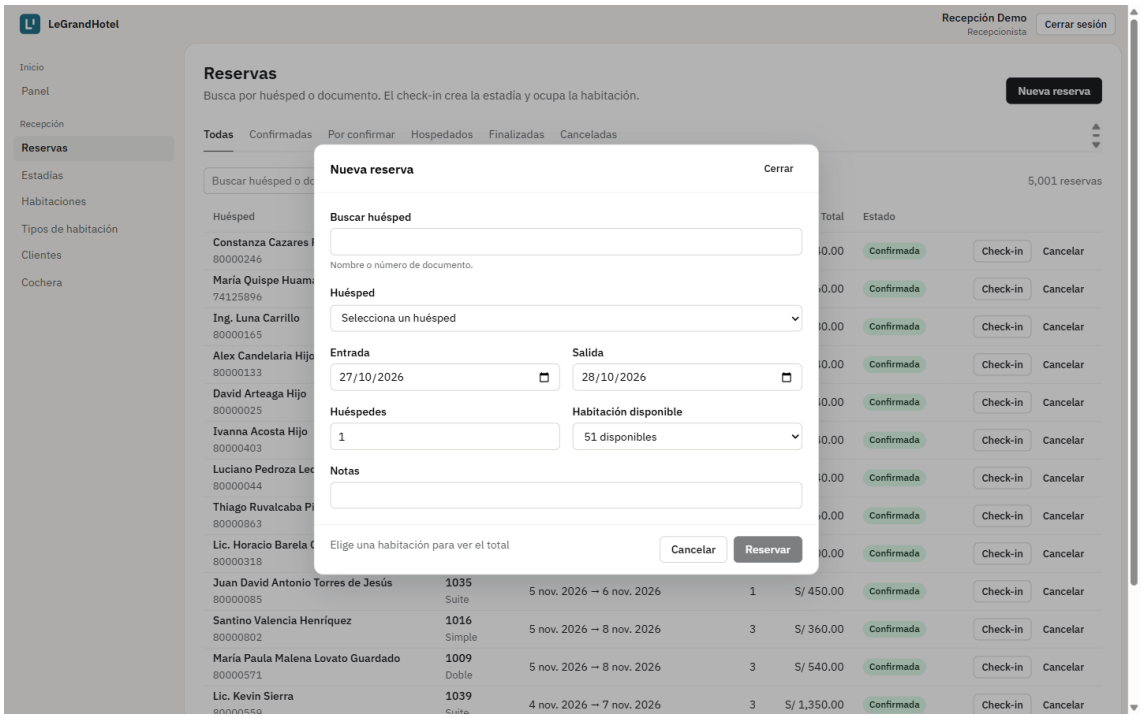


Figura 7. Nueva reserva.

Se busca al huésped, se eligen las fechas y solo aparecen las habitaciones libres para esas fechas y esa cantidad de personas. El total lo calcula el servidor (noches × precio); si otra persona toma la misma habitación al mismo tiempo, la API responde 409.

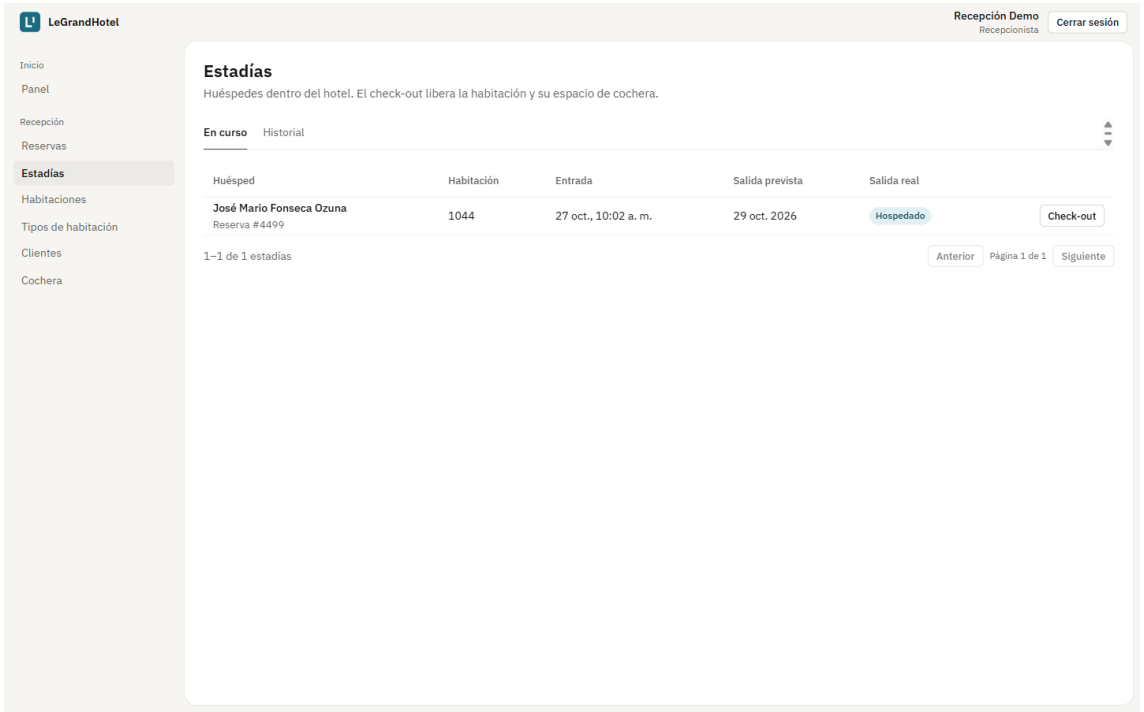


Figura 8. Estadías.

Huéspedes que están dentro del hotel y su historial. El check-out cierra la estadía, pasa la habitación a limpieza y libera su espacio de cochera.

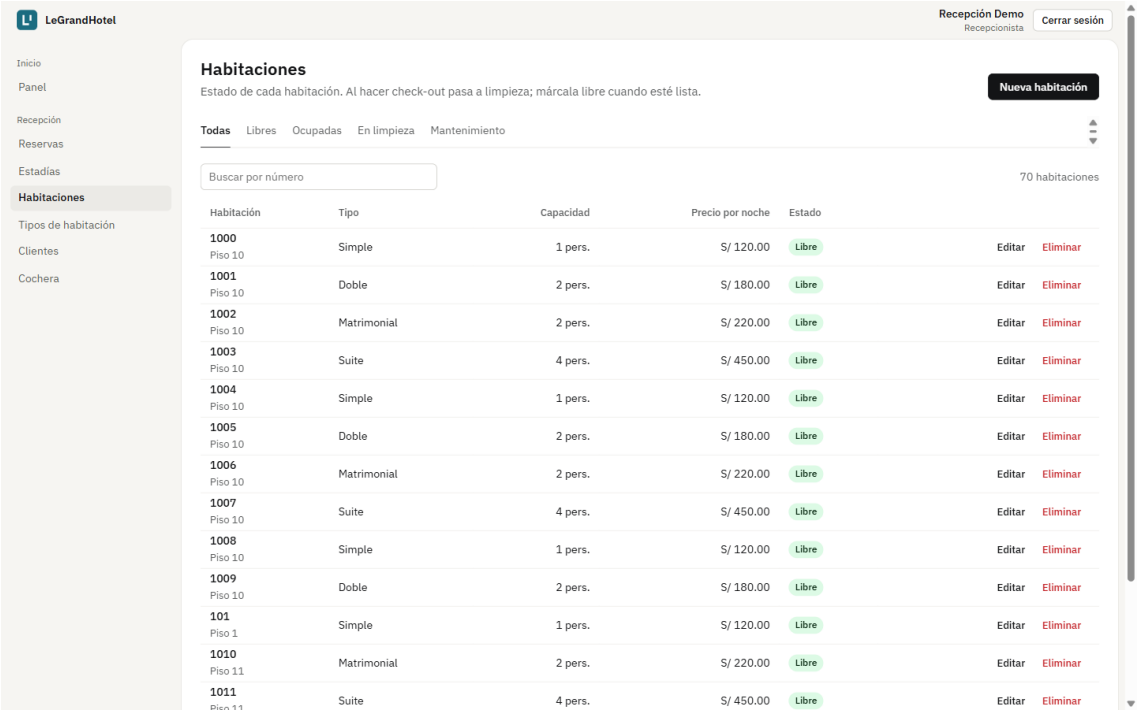


Figura 9. Habitaciones.

Habitaciones por estado y tipo. Recepción marca como «lista» una habitación después de la limpieza.

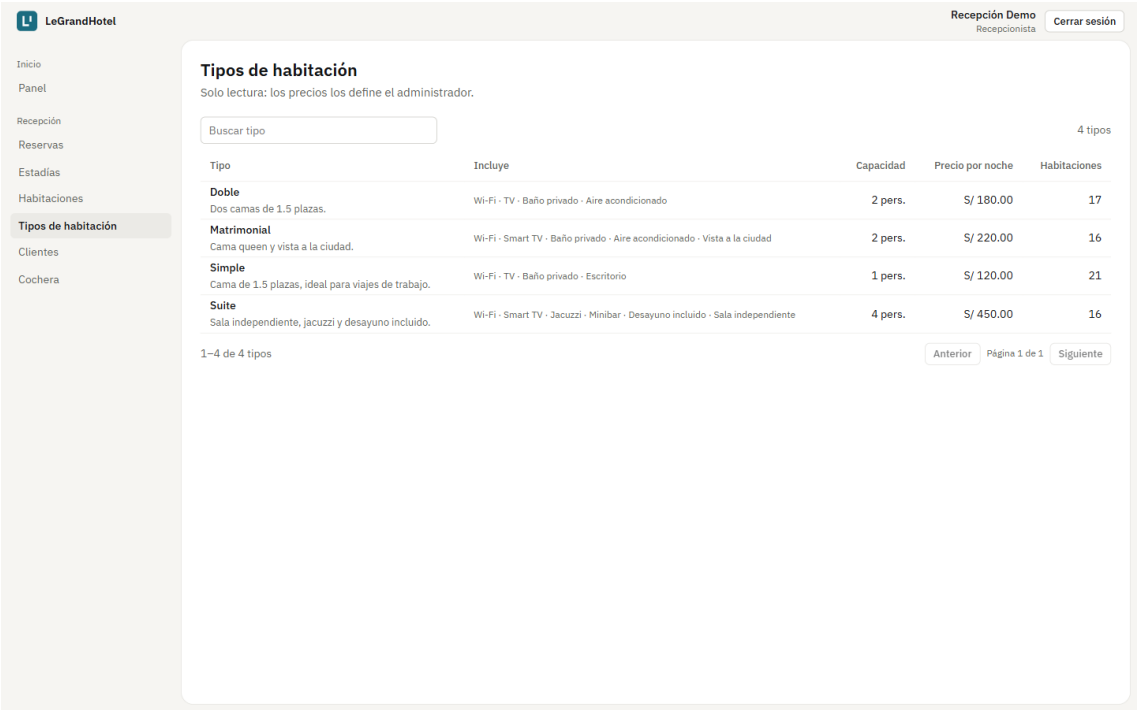


Figura 10. Tipos de habitación (solo lectura para Recepción).

Recepción consulta el precio por noche, la capacidad y lo que incluye cada tipo para informar al huésped, pero no puede cambiarlos.

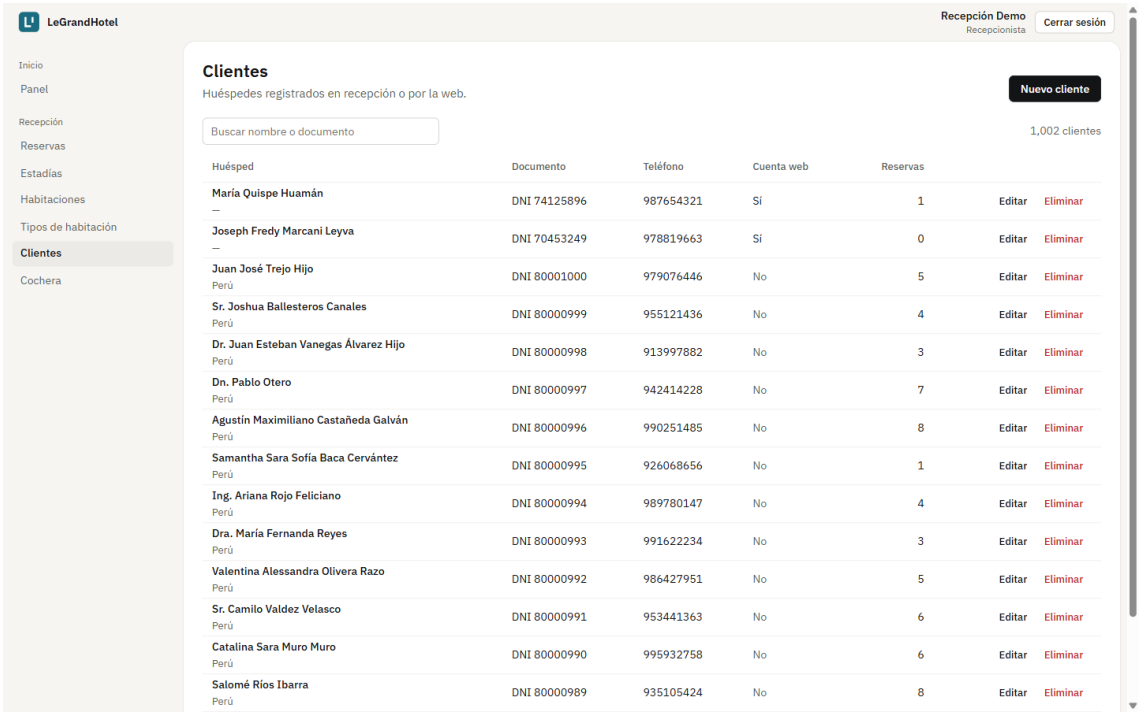


Figura 11. Clientes (huéspedes).

Fichas de huéspedes con documento, teléfono, si tienen cuenta web y cuántas reservas tienen. Solo se puede eliminar un huésped sin reservas.

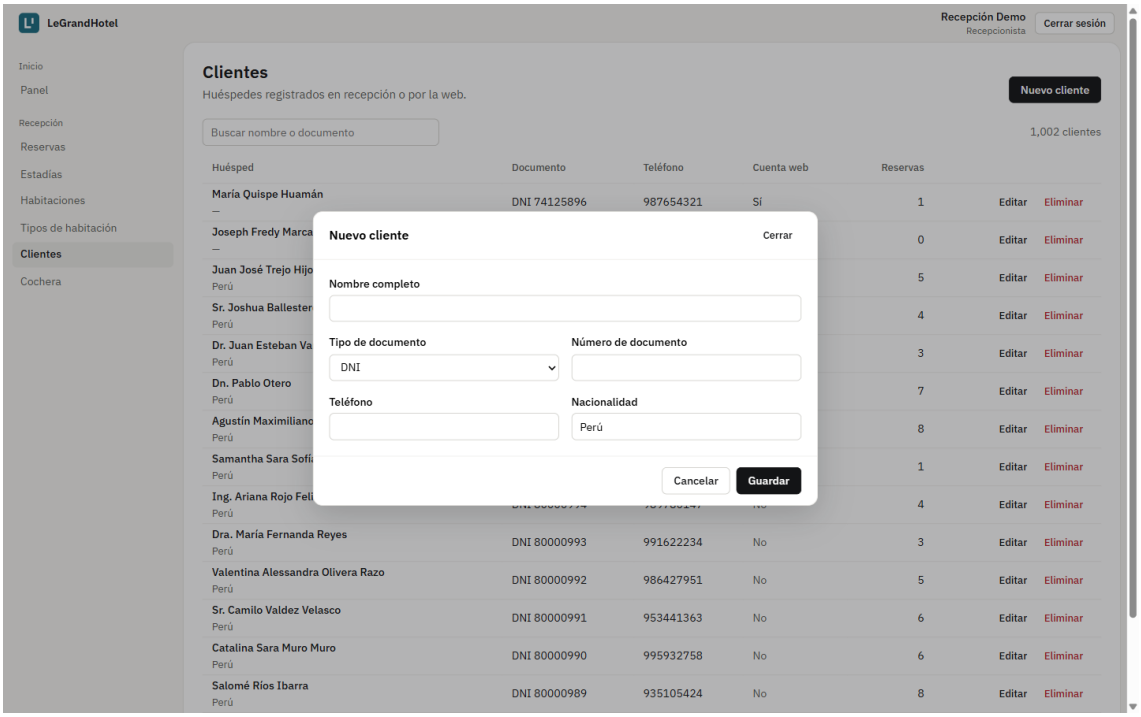


Figura 12. Nuevo cliente.

Registro de un huésped que llega o llama: tipo y número de documento (único), nombre, teléfono y nacionalidad.

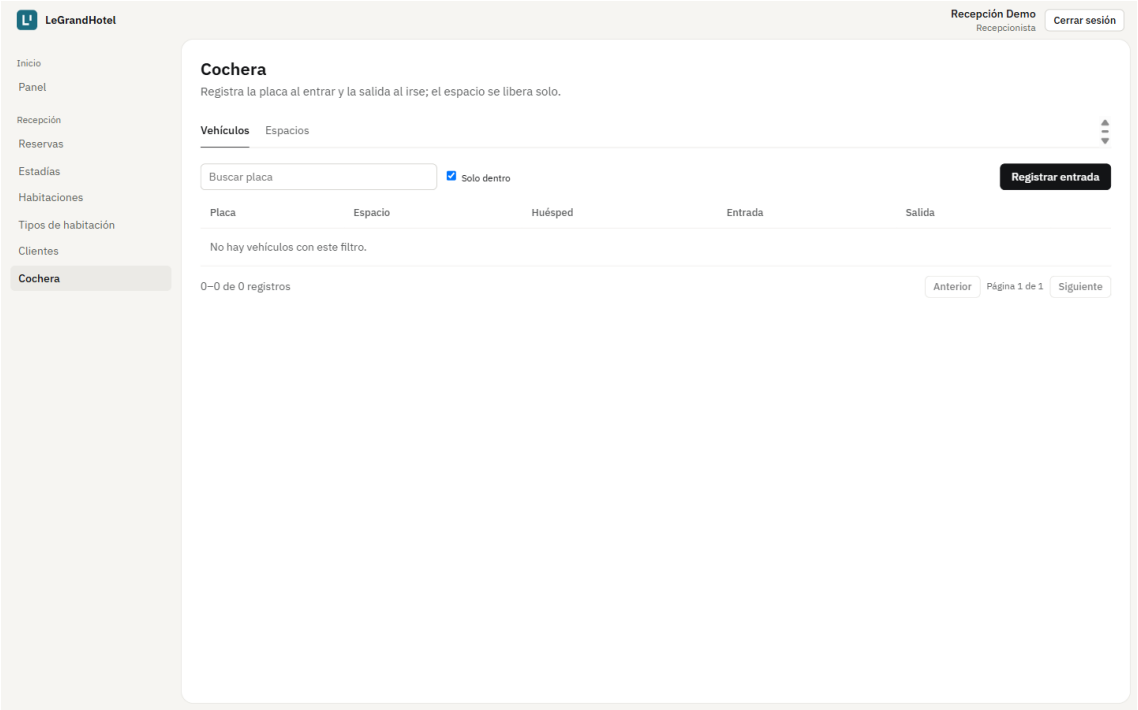


Figura 13. Cochera.

Vehículos dentro y espacios libres. Cada entrada puede asociarse a una estadía.

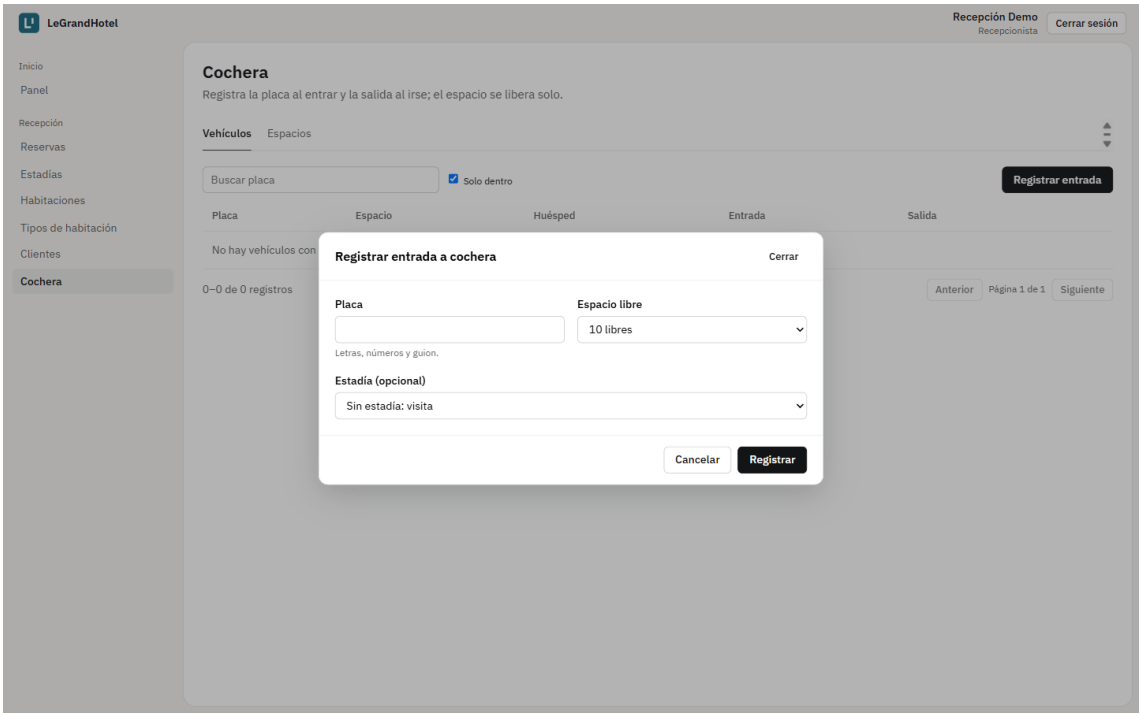


Figura 14. Registrar entrada a cochera.

Se anota la placa y el espacio; si el espacio ya está ocupado, la API responde 409 PARKING_SPOT_NOT_FREE.

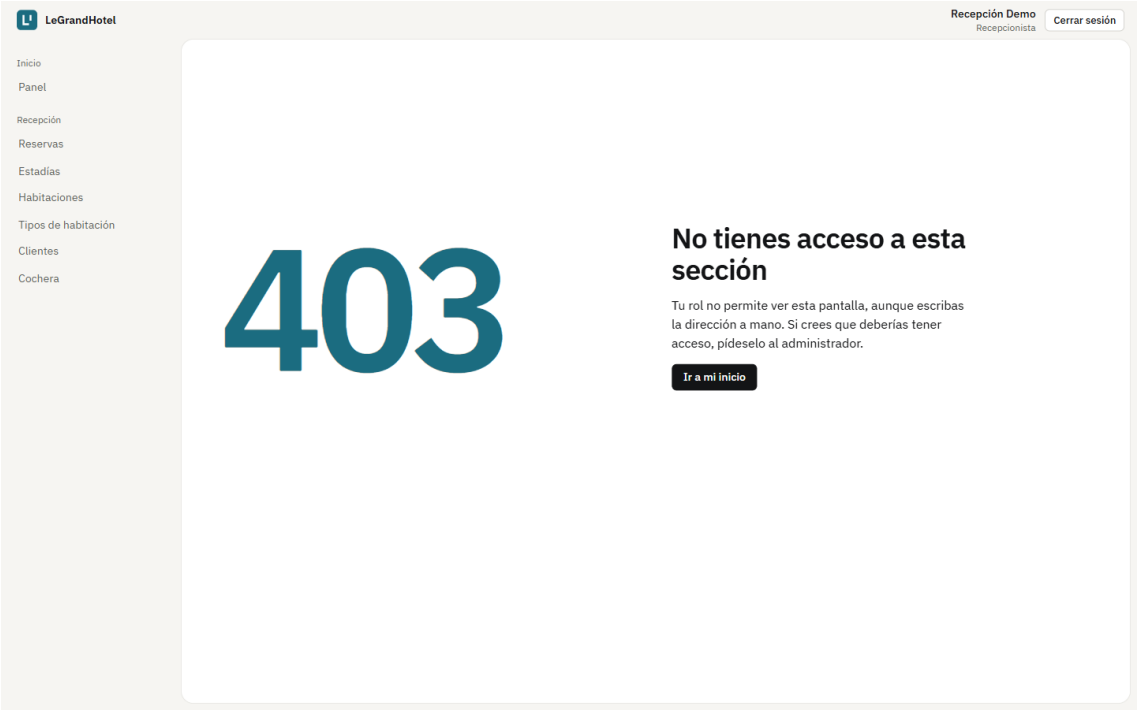


Figura 15. Recepción intentando abrir Inventario.

Si alguien escribe a mano la dirección de una pestaña que no le corresponde, ve la pantalla 403. La API también responde 403, así que no hay forma de saltarse el permiso.

4.2 Almacenero

Controla los insumos del hotel: lo que entra, lo que sale y lo que hay que reponer.

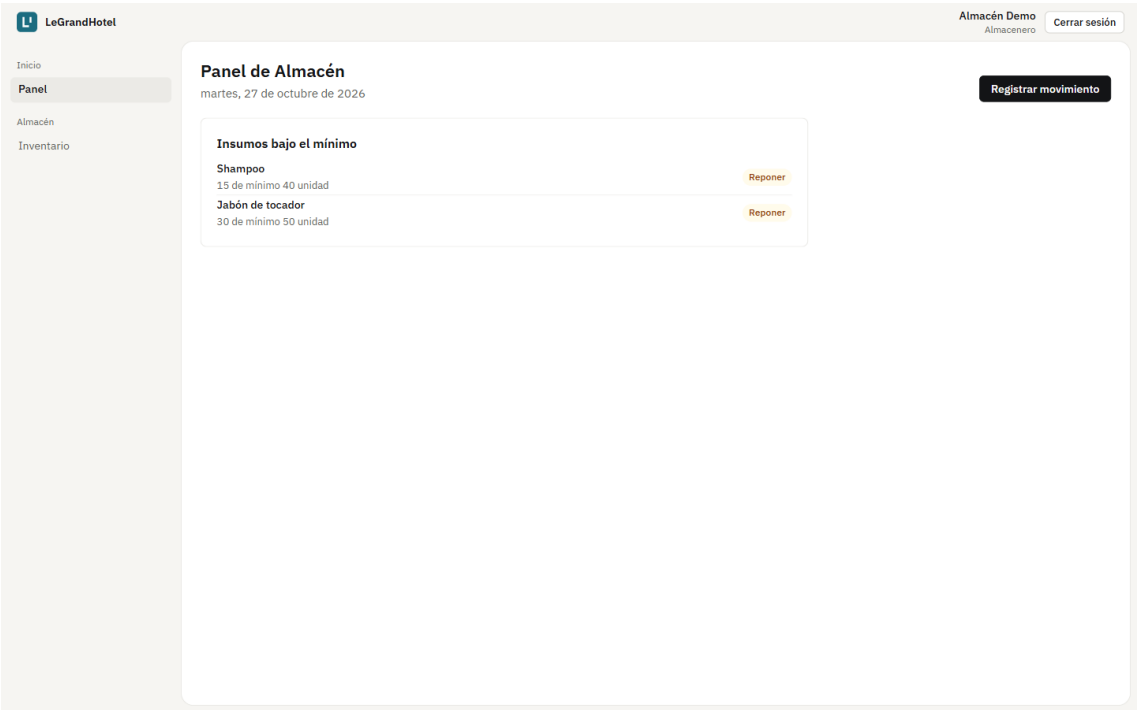


Figura 16. Panel del Almacén.

Muestra los insumos que están por debajo del mínimo para reponerlos a tiempo.

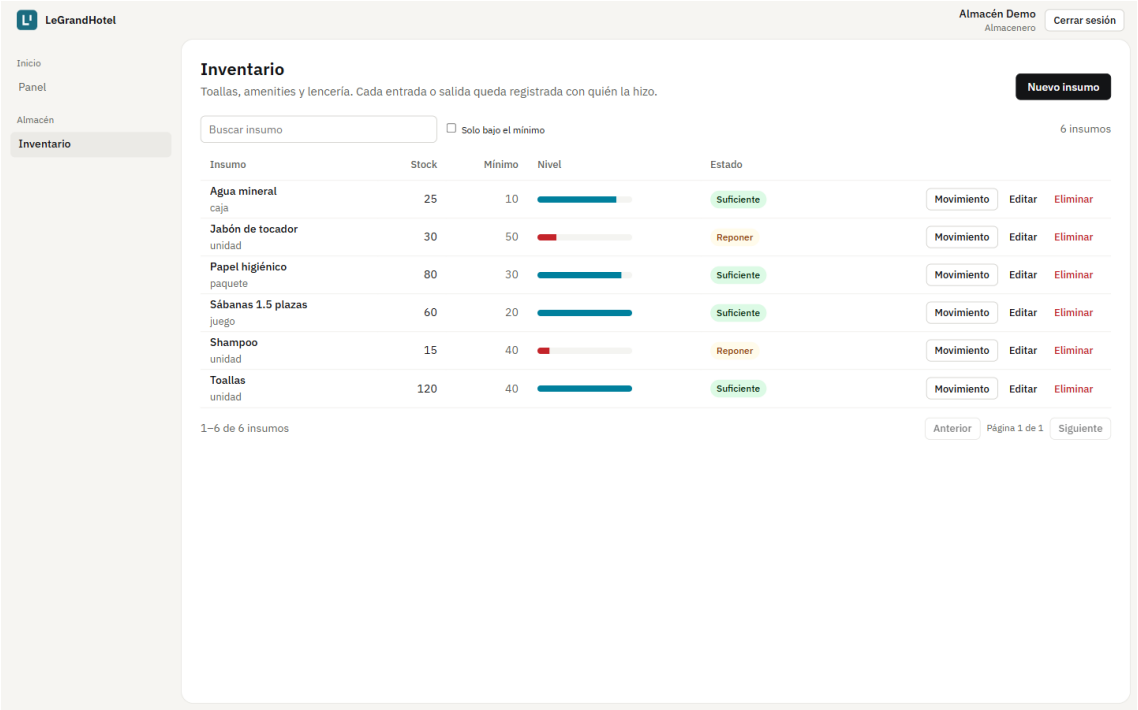


Figura 17. Inventario.

Cada insumo con su stock, el mínimo y una barra de nivel. Se puede filtrar solo los que están bajo el mínimo.

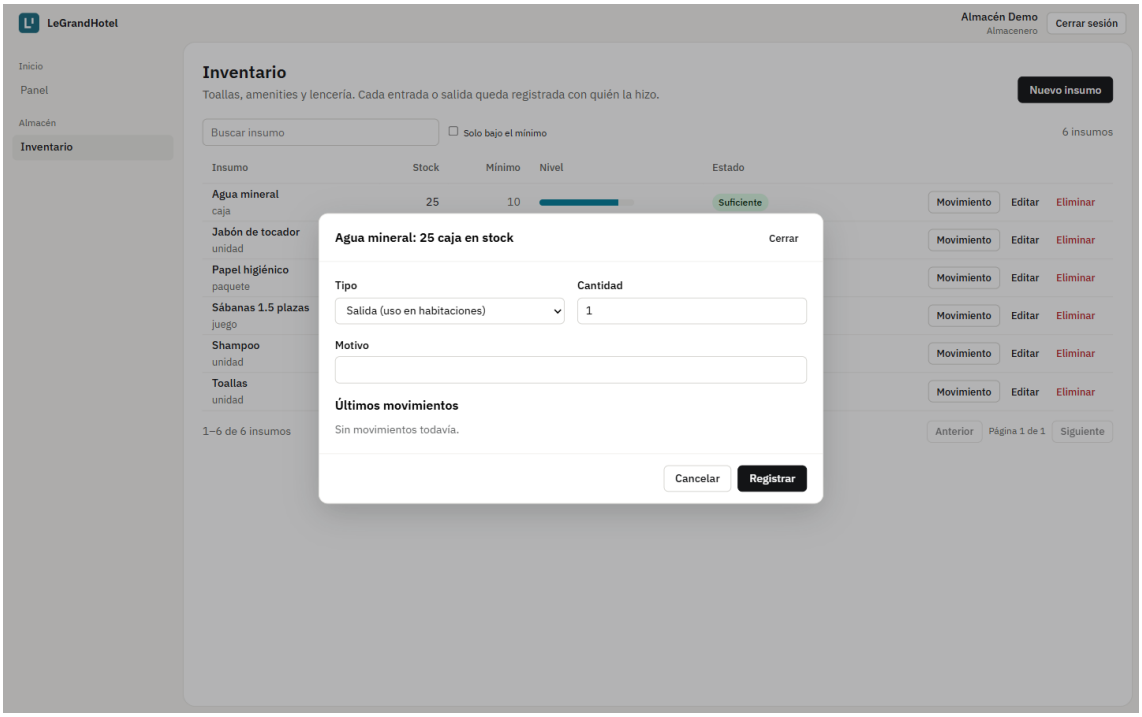


Figura 18. Registrar movimiento.

Entrada o salida con cantidad y motivo; abajo se ven los últimos movimientos. La API no deja sacar más de lo que hay (409 INSUFFICIENT_STOCK).

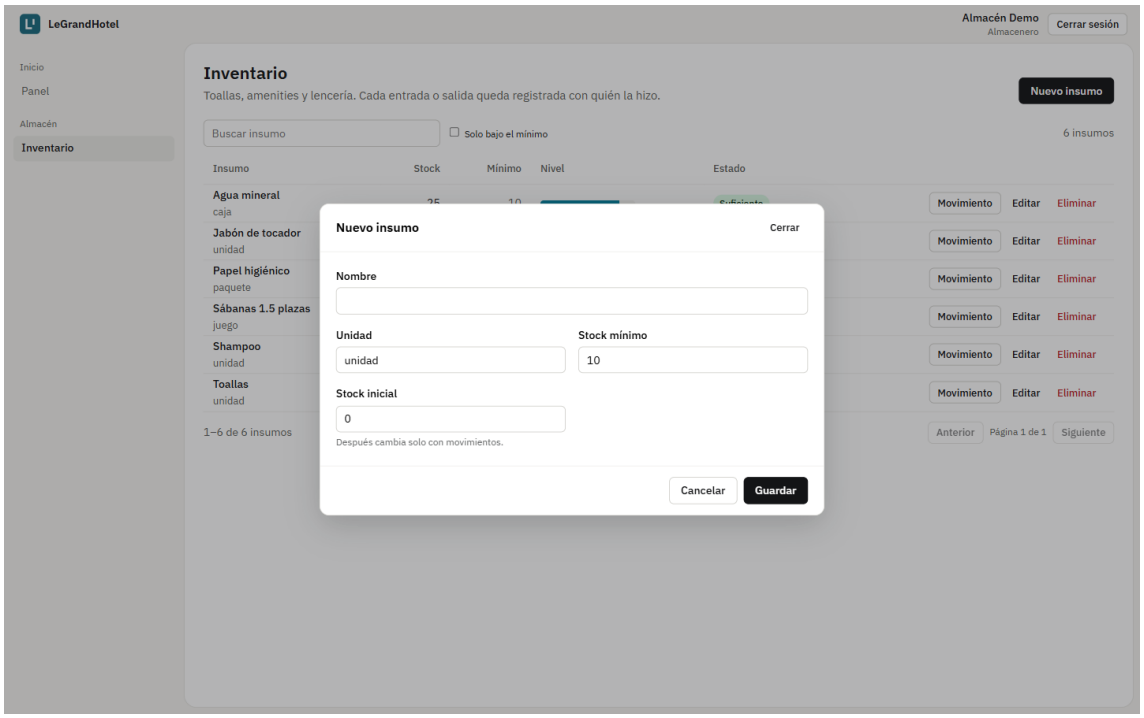


Figura 19. Nuevo insumo.

Nombre, unidad, stock inicial y mínimo. Después el stock solo cambia con movimientos, para que todo quede registrado.

4.3 Administrador

Es el responsable del sistema: tiene todas las pestañas del personal, define tipos y precios, gestiona al personal y sus cuentas, y revisa reportes y auditoría.

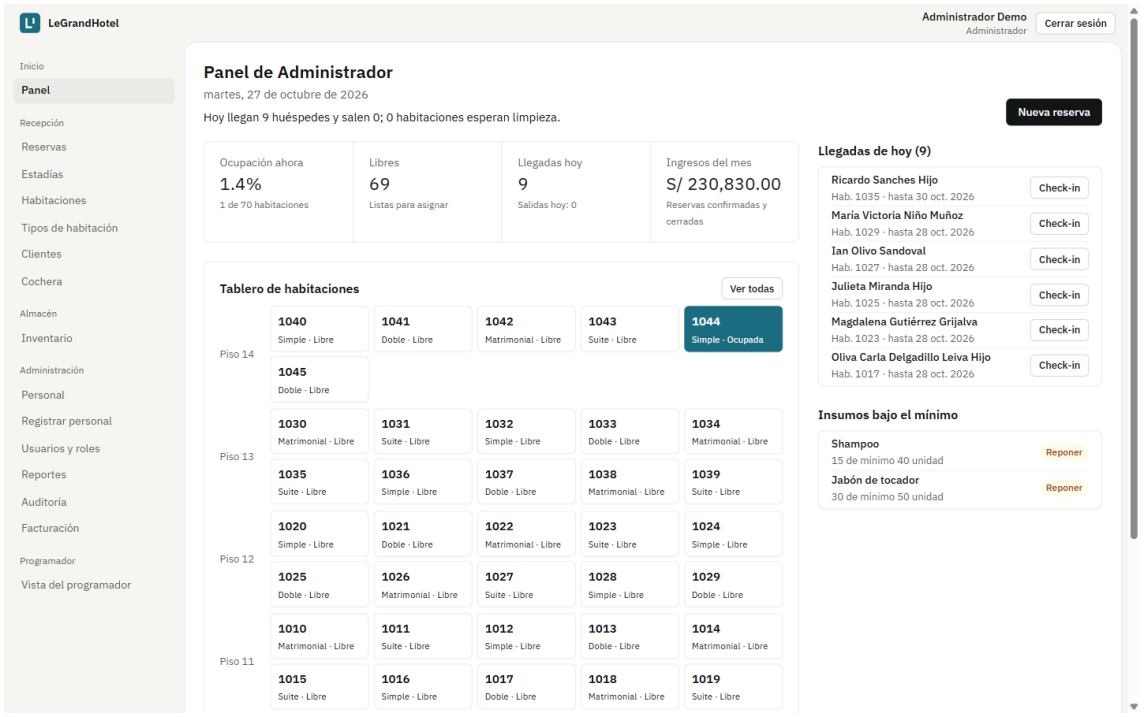


Figura 20. Panel del Administrador con el menú completo.

El mismo resumen del día, con todos los grupos del menú: Inicio, Recepción, Almacén, Administración y Programador.

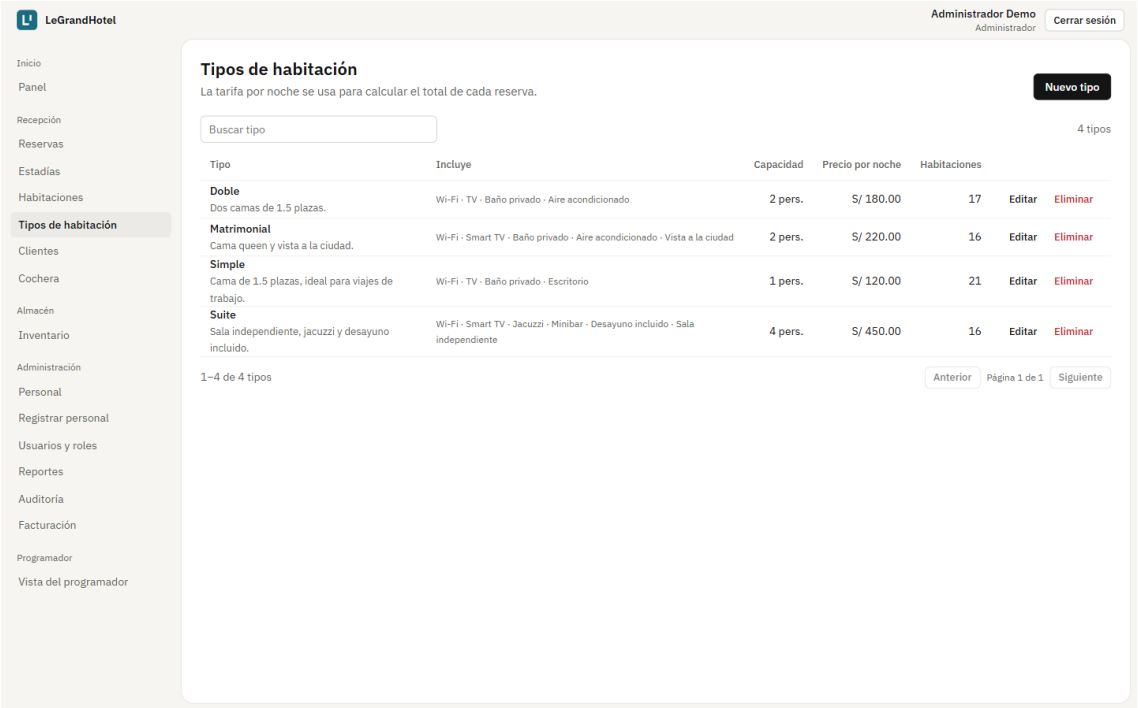


Figura 21. Tipos de habitación (Administrador).

A diferencia de Recepción, el Administrador puede crear, editar y eliminar tipos.

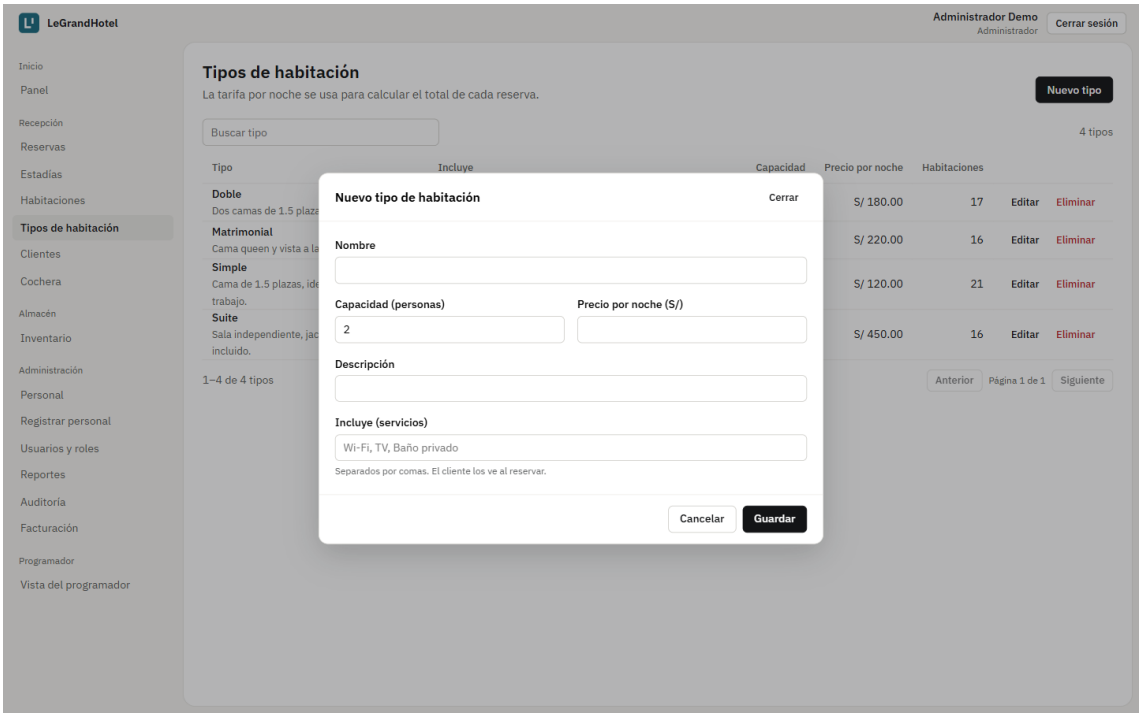


Figura 22. Nuevo tipo de habitación.

Nombre, capacidad, precio por noche, descripción y lo que incluye (separado por comas). Eso es lo que verá el cliente al reservar.

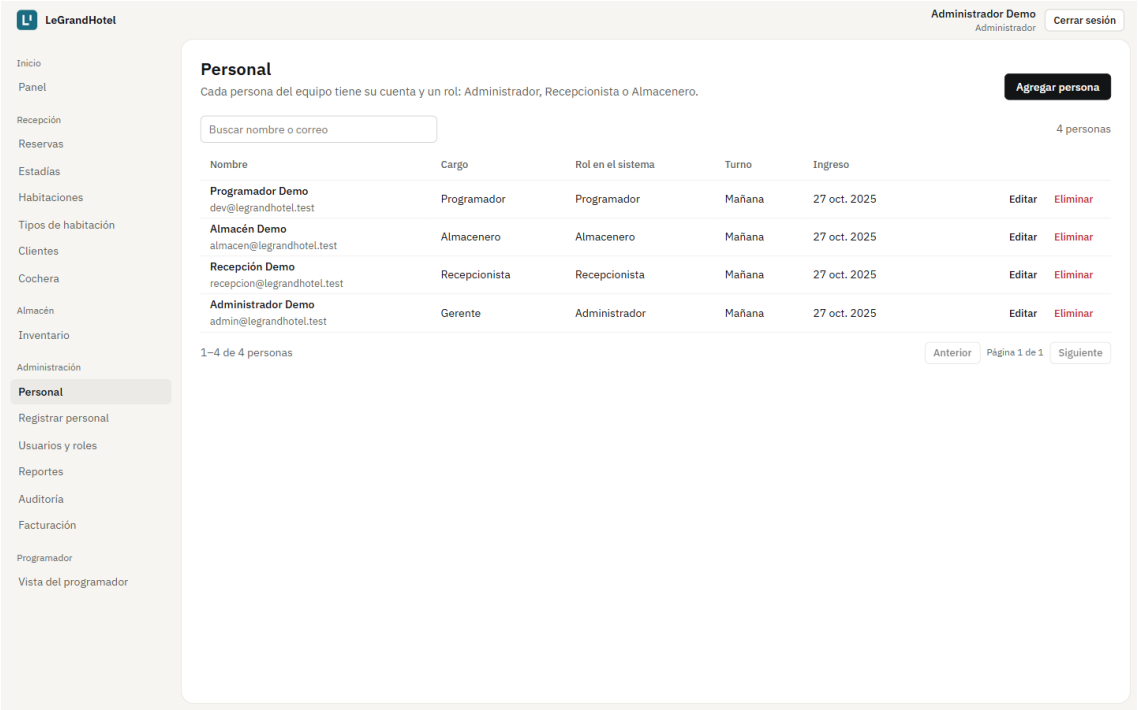


Figura 23. Personal.

Fichas del personal con cargo, turno y fecha de ingreso, cada una ligada a su cuenta.

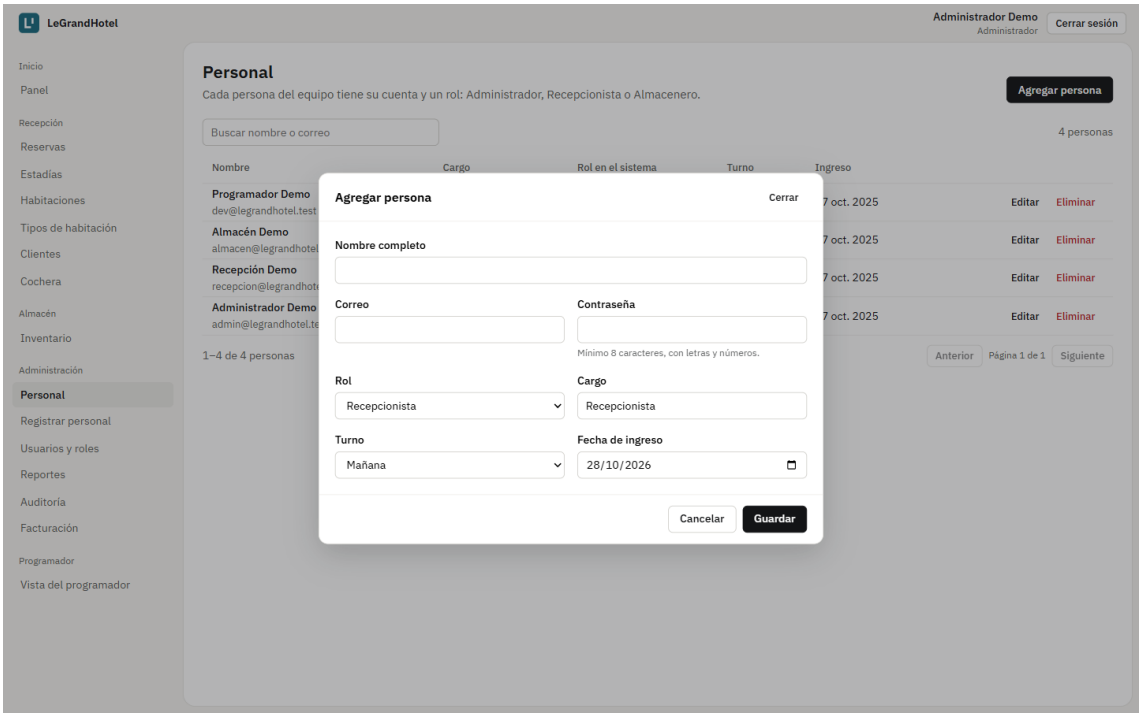


Figura 24. Agregar persona del personal.

Crea la ficha y la cuenta en un solo paso. El rol Programador no aparece porque el hotel tiene uno solo.

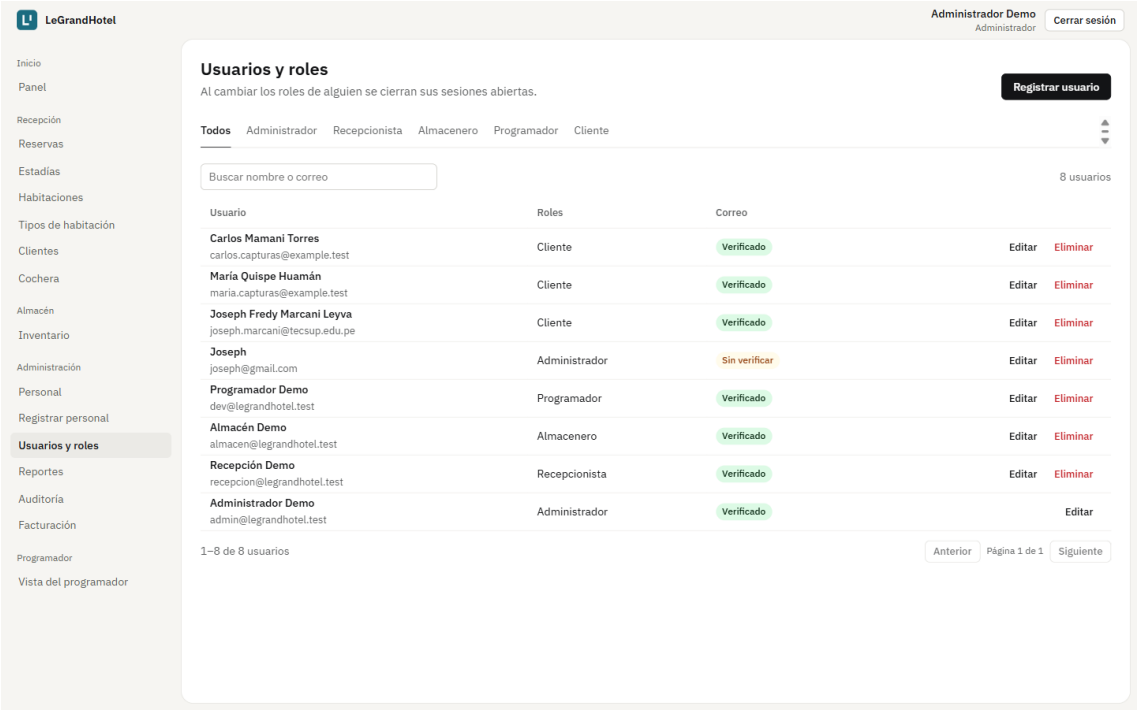


Figura 25. Usuarios y roles.

Todas las cuentas con su rol y si verificaron el correo; se filtran por rol y se busca por nombre o correo.

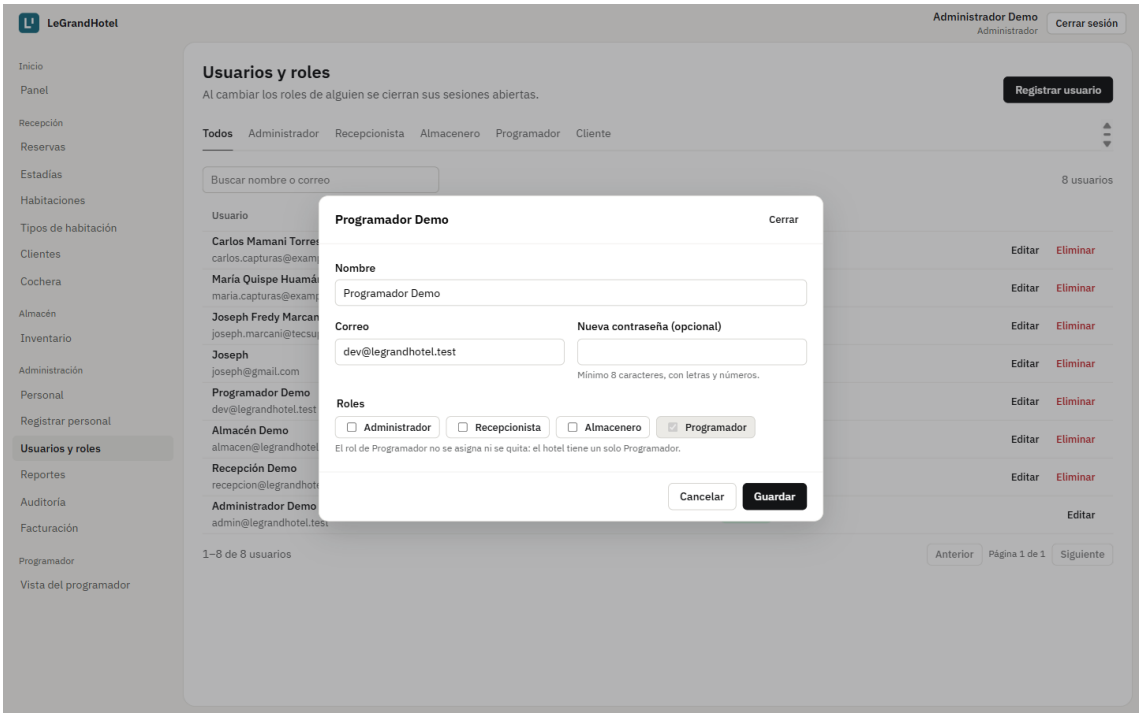


Figura 26. Editar la cuenta del Programador.

La casilla Programador está bloqueada: no se puede dar ese rol a otra cuenta ni quitárselo al Programador. Sí se pueden editar su nombre, su correo y su contraseña.

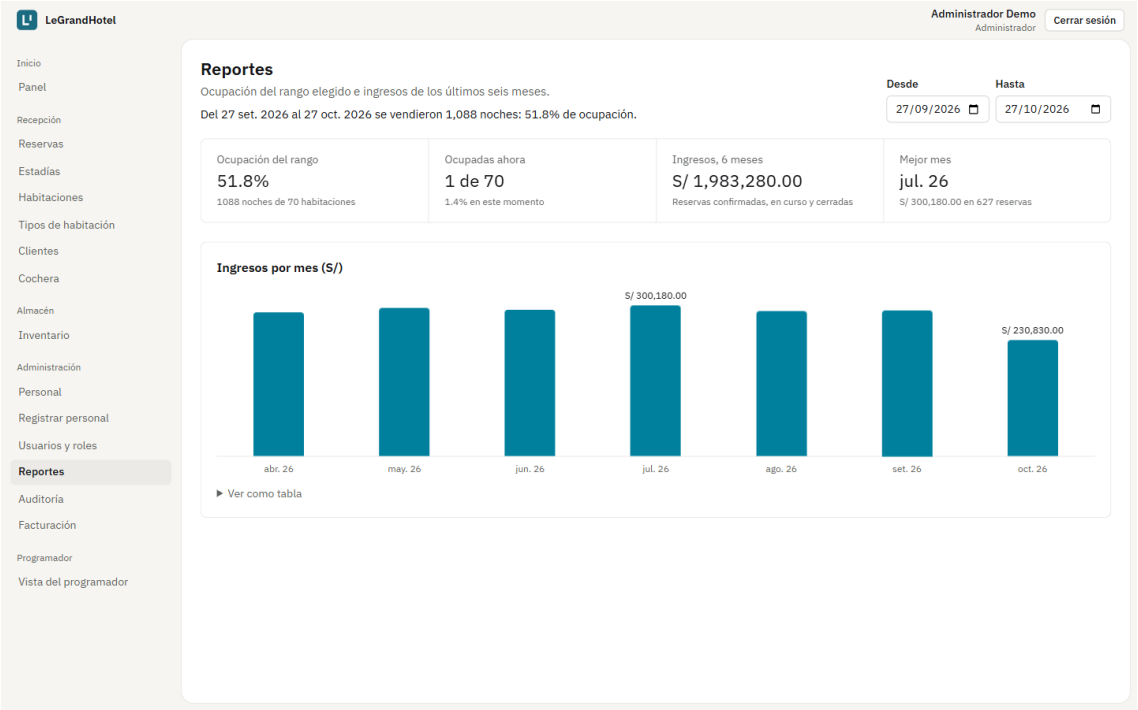


Figura 27. Reportes.

Ocupación de un rango de fechas e ingresos de los últimos 6 meses en un gráfico de barras.

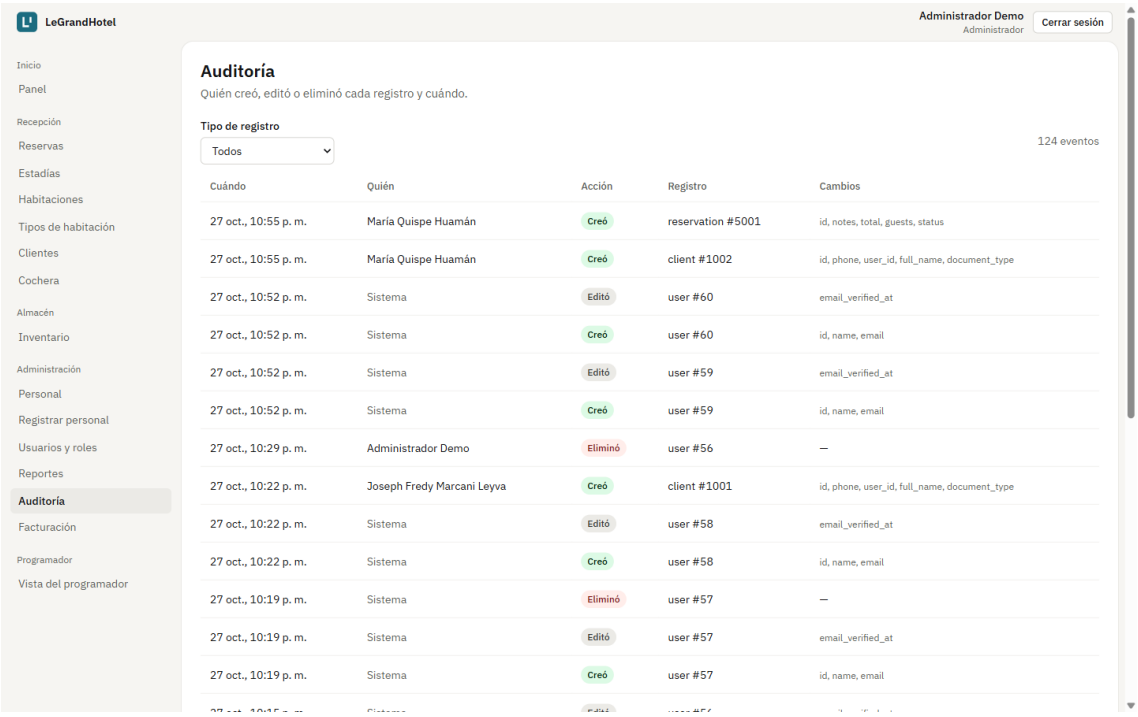


Figura 28. Auditoría.

Quién creó, editó o eliminó cada registro, cuándo y desde qué IP, con los valores antes y después del cambio.

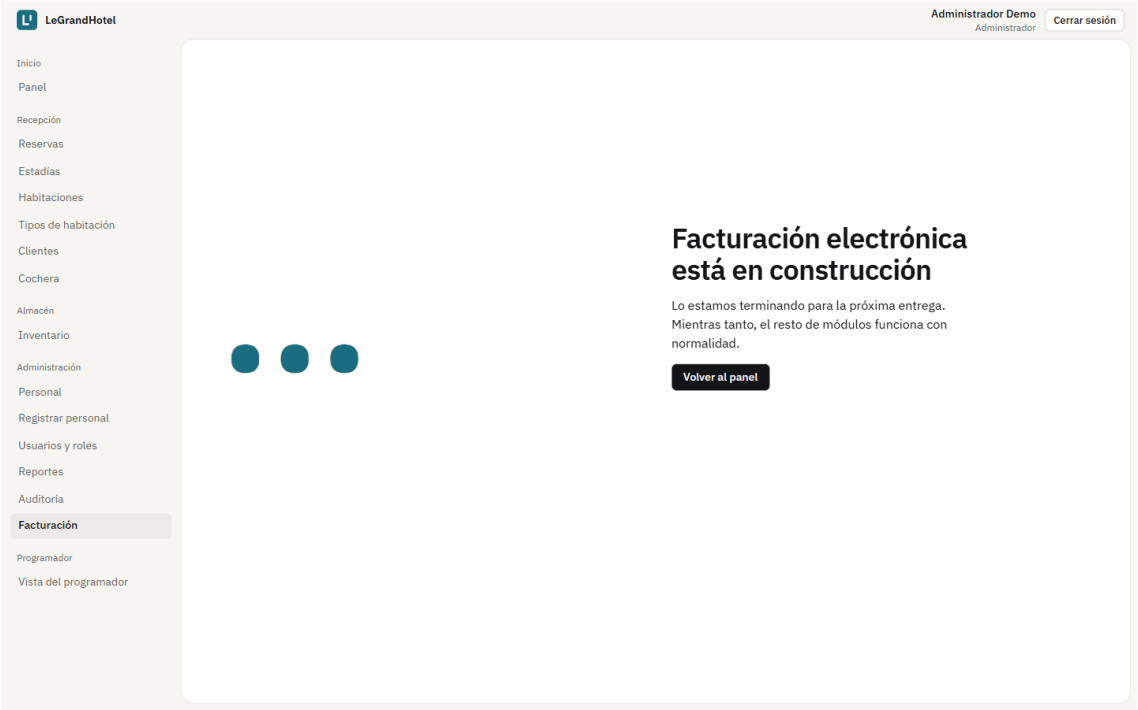


Figura 29. Facturación.

Módulo previsto que todavía no está construido: muestra la pantalla «En construcción» en vez de un error.

4.4 Programador

Da soporte técnico: ve el estado de la API sin entrar al servidor y registra cuentas del personal cuando hace falta.

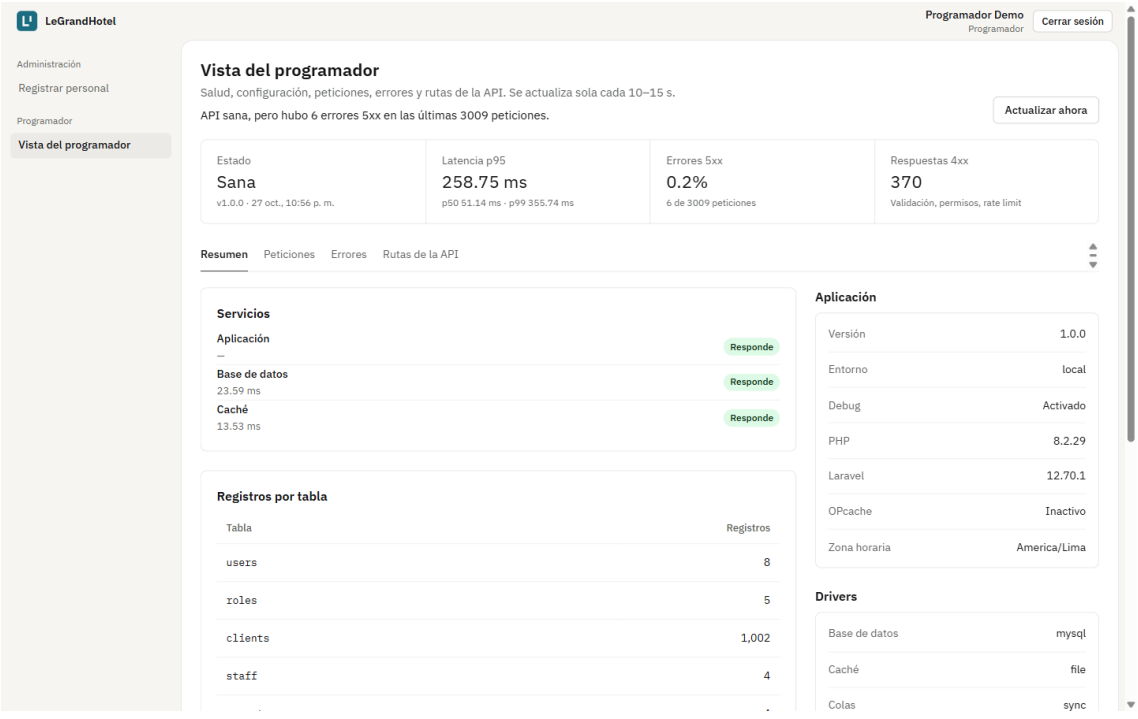


Figura 30. Vista del programador · Resumen.

Estado de la API, latencias p50, p95 y p99, porcentaje de errores 5xx y respuestas 4xx, y la configuración (sin mostrar secretos).

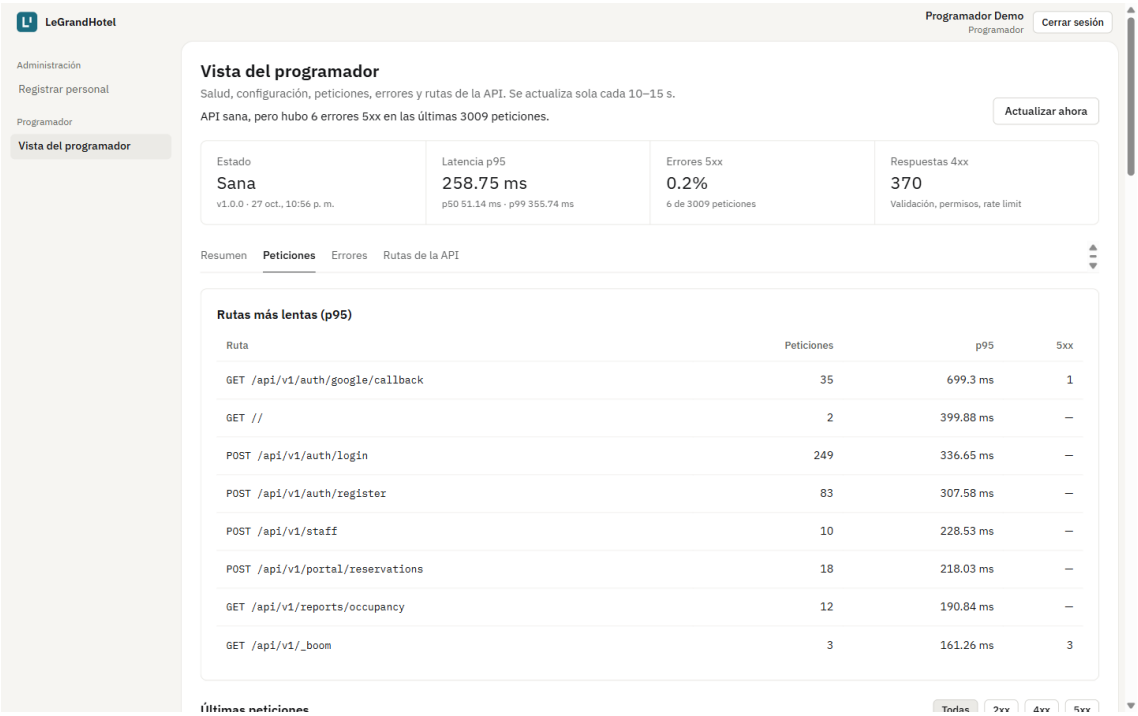


Figura 31. Vista del programador · Peticiones.

Últimas peticiones con su código y duración, y las rutas más lentas por p95.

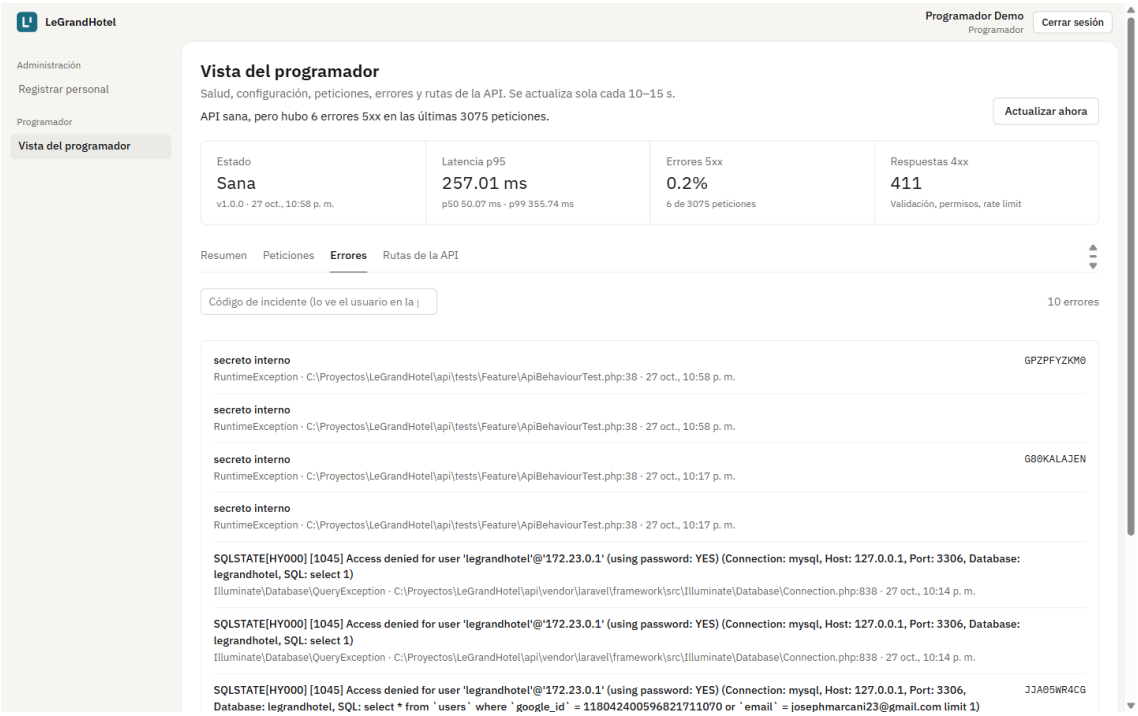


Figura 32. Vista del programador · Errores.

Errores del servidor, buscables por el código de incidente que ve el usuario en la pantalla 500. Aquí aparece el incidente real JJA05WR4CG de la sección 6.

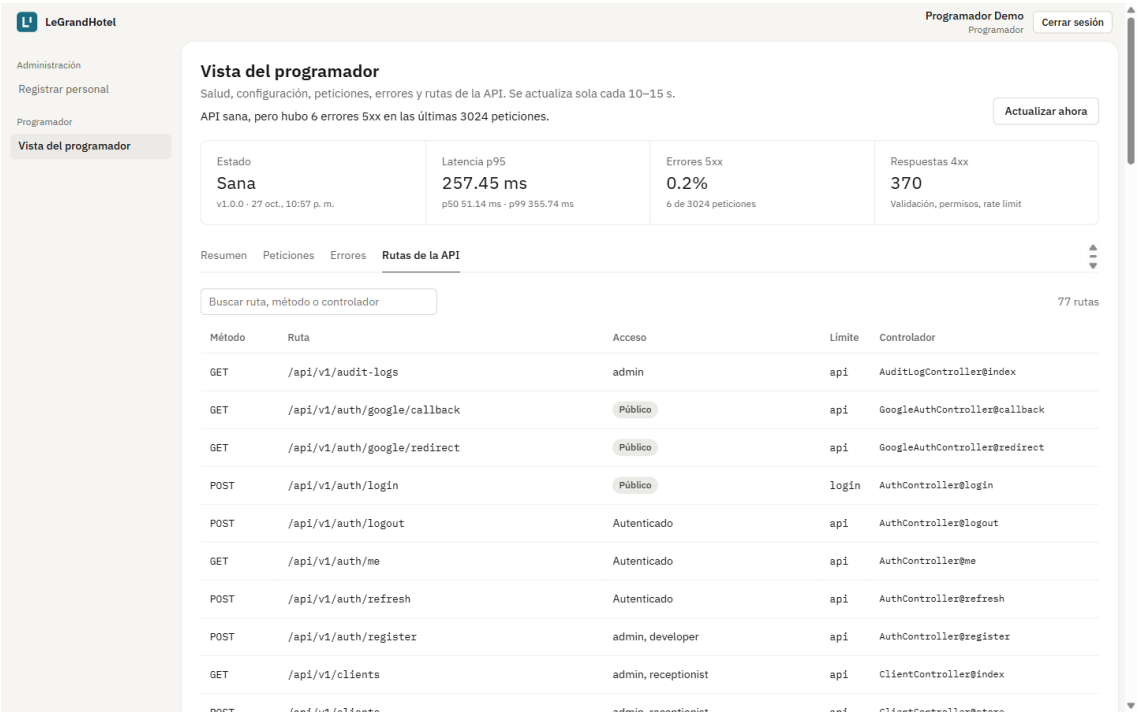


Figura 33. Vista del programador · Rutas de la API.

Las rutas de la API con su método, si piden sesión, los roles que la pueden usar y su límite de peticiones.

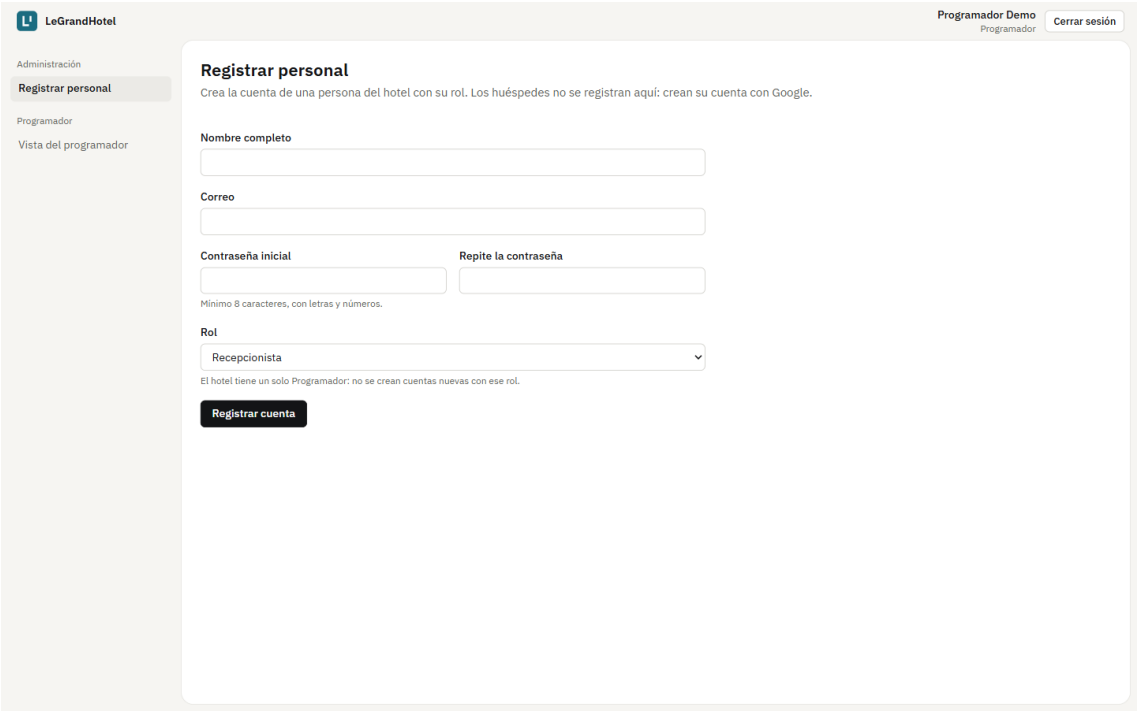


Figura 34. Registrar personal (Programador).

El Programador también crea cuentas de Recepción, Almacén y Administración, pero no de Programador.

4.5 Cliente

Es el huésped con cuenta propia: entra con Google, completa su DNI y teléfono una vez y reserva en línea. No ve nada del panel interno.

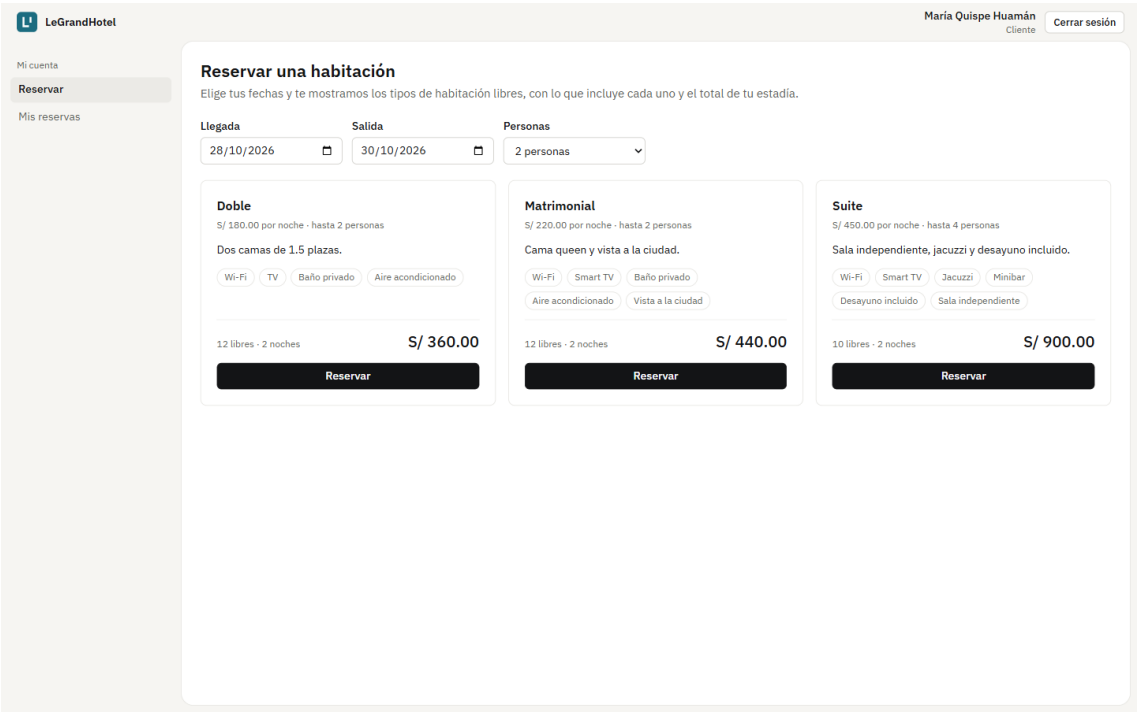


Figura 35. Reservar una habitación.

Elige llegada, salida y personas. Ve cada tipo con lo que incluye, cuántas habitaciones quedan libres y el total de su estadía. Los tipos que no alcanzan para esa cantidad de personas no aparecen.

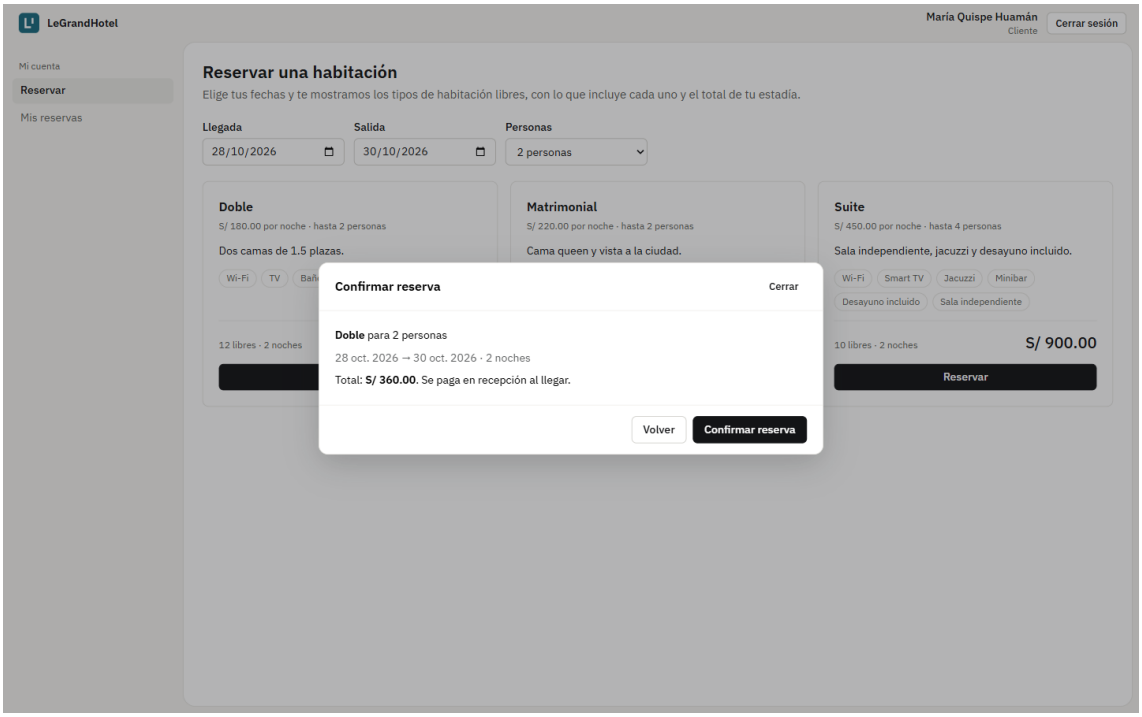


Figura 36. Confirmar la reserva.

Resumen con fechas, noches y total antes de confirmar. Al confirmar, el hotel le asigna una habitación libre de ese tipo; si otra persona la toma en ese instante, el sistema prueba con la siguiente.

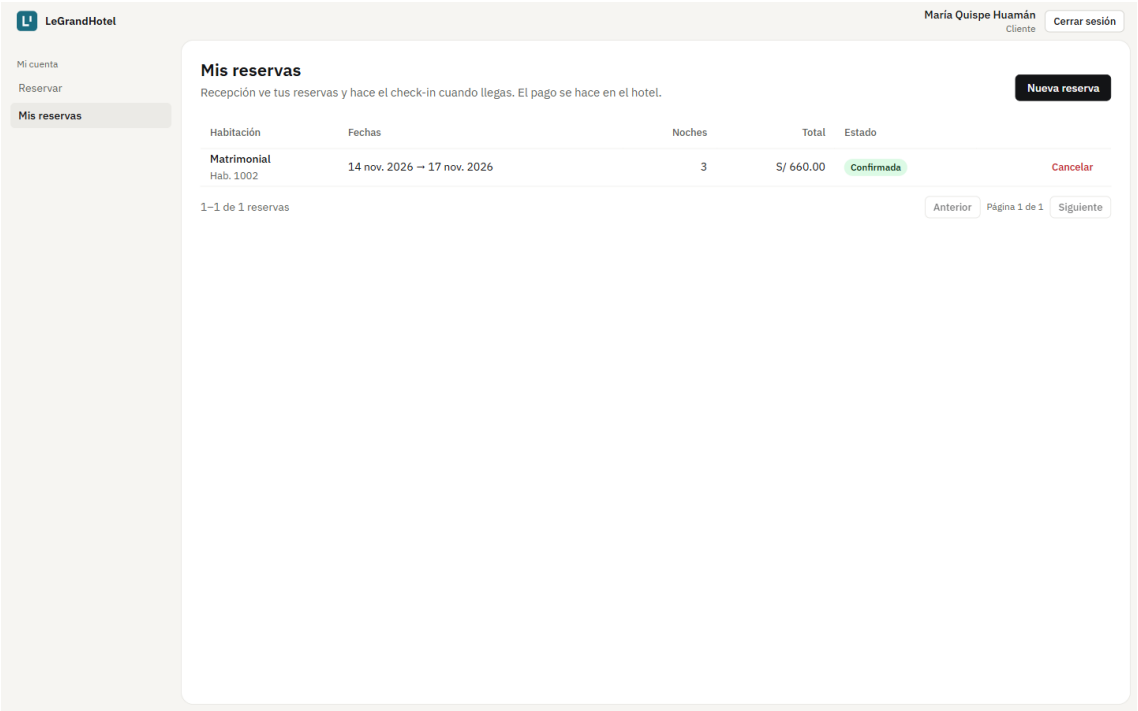


Figura 37. Mis reservas.

Sus reservas con habitación, fechas, total y estado. Puede cancelar una reserva confirmada mientras falte al menos un día para la llegada.

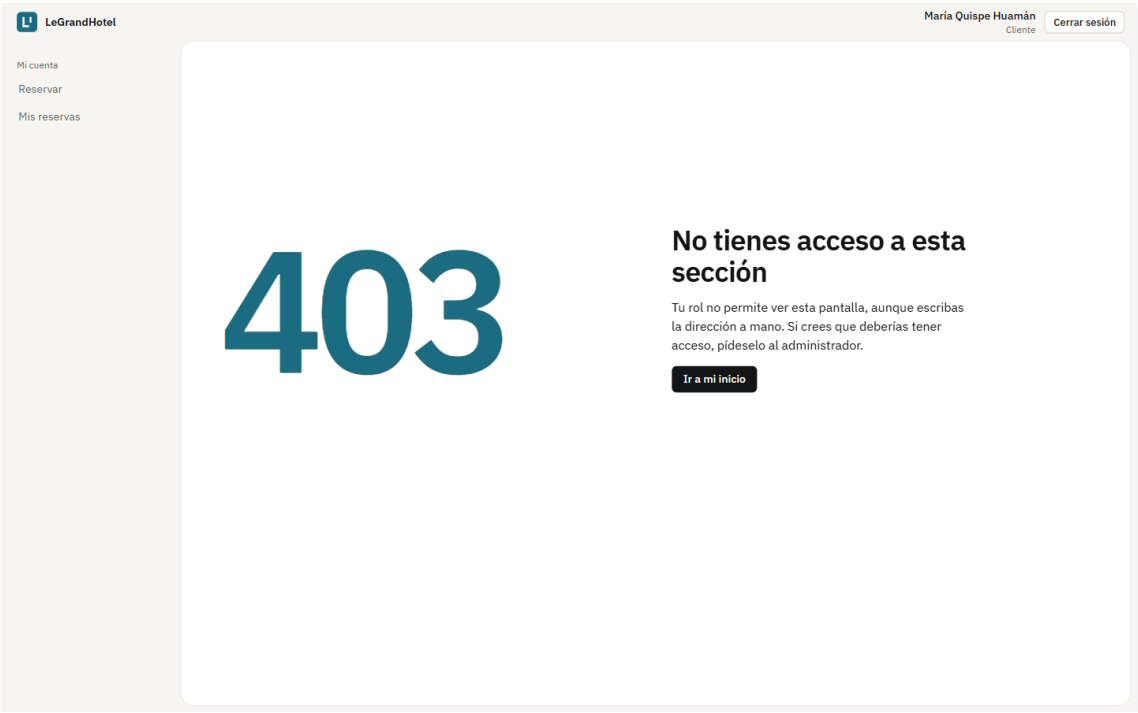


Figura 38. Cliente intentando abrir el panel del personal.

Un cliente no puede entrar a ninguna pestaña del personal: ve la pantalla 403 y la API responde 403.

5. Pantallas de error

Cada respuesta de error de la API tiene su pantalla, con el mismo estilo de la página, un mensaje claro y una salida (volver al inicio, recargar o iniciar sesión). Las decide un solo interceptor en web/src/lib/api.ts, así se comportan igual en todas las pestañas.

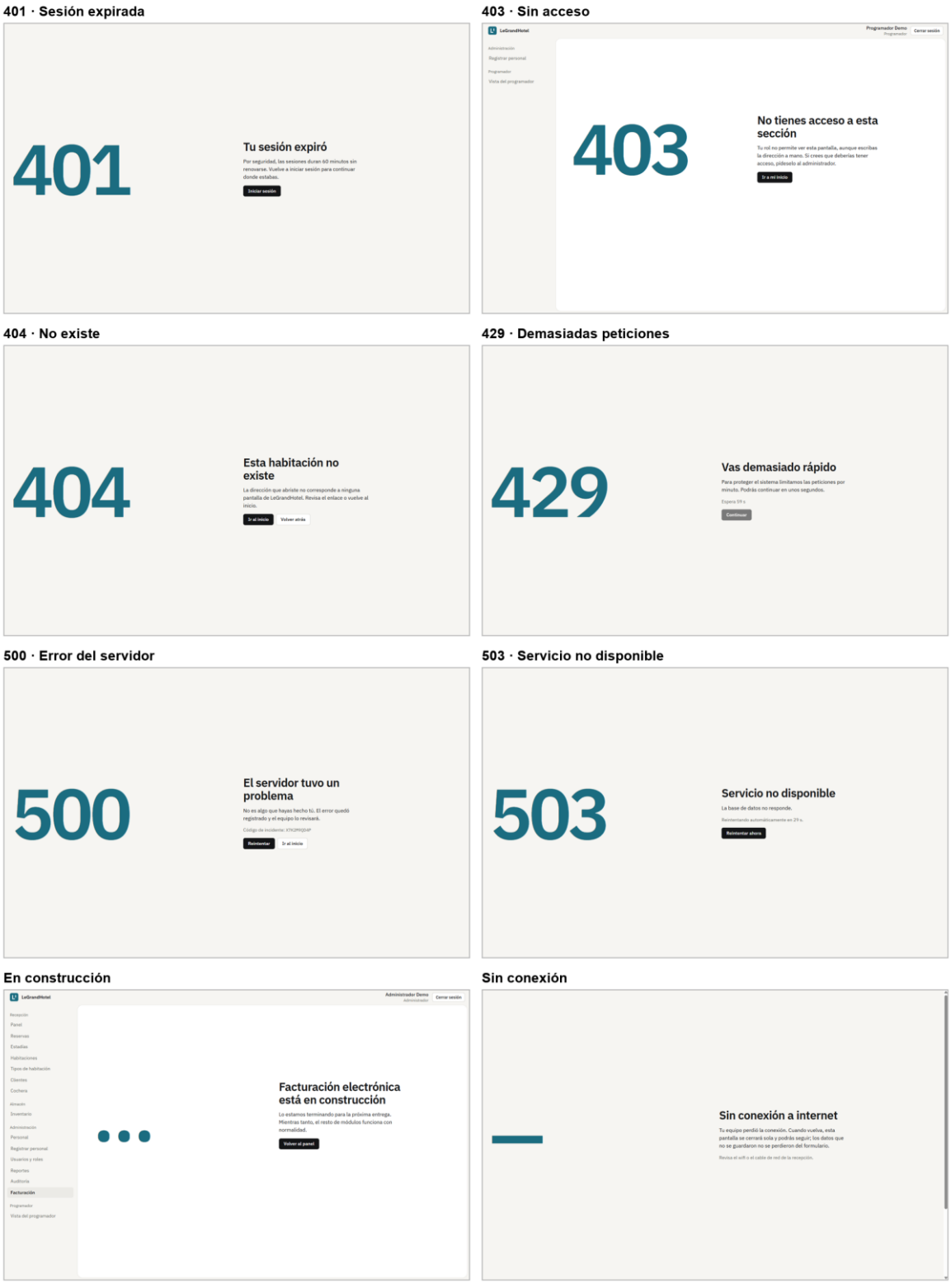


Figura 39. Pantallas 401, 403, 404, 429, 500, 503, en construcción y sin conexión.

Pantalla	Cuándo aparece
401 · Sesión expirada	El token venció (60 minutos) o se cerró la sesión en otro lugar
403 · Sin acceso	El rol no tiene permiso para esa pestaña o esa acción
404 · No encontrada	La dirección no existe o el registro fue eliminado
429 · Demasiadas peticiones	Se superó el límite (por ejemplo 5 logins por minuto); muestra cuánto esperar

Pantalla	Cuándo aparece
500 · Error del servidor	Algo falló; muestra el código de incidente para buscarlo en la Vista del programador
503 · No disponible	La API no responde; reintenta sola cada 30 segundos
En construcción	Módulo previsto pero aún no construido (Facturación)
Sin conexión	El equipo perdió internet; vuelve sola al reconectarse

6. Registro de incidentes

Toda caída o falla, aunque sea en el entorno local o en pruebas, se anota el mismo día en docs/incidentes.md con su causa y su solución. El registro tiene ahora 8 incidentes; 6 son del 27 de octubre, al repetir las pruebas de carga, configurar el inicio con Google y revisar la Vista del programador.

Fecha	Qué falló	Causa	Solución	Tiempo
2026-10-26	El login devolvía 500 INTERNAL_ERROR con código de incidente XG1R7UASWT	php artisan serve lanza procesos PHP hijos sin la extensión pdo_sqlite, cargada solo con -d en el proceso padre ("could not find driver")	Se arranca el servidor local con php -d extension=pdo_sqlite -S localhost:8000 -t api/public api/public/index.php (.claude/launch.json). El error salió como JSON sin trazas, como exige el plan	~2 min
2026-10-26	El frontend mostró la pantalla 503 "No se pudo contactar con el servidor"	Durante el cambio anterior el servidor PHP apuntaba a un server.php que buscaba index.php en la carpeta equivocada	Se usa api/public/index.php como router del servidor. La pantalla 503 reintentó sola cada 30 s	~1 min
2026-10-27	Primera serie de k6: 100% de peticiones fallidas	El servidor PHP escuchaba solo en IPv6 (::1) y k6 resolvía localhost a IPv4 (127.0.0.1)	BASE_URL=http://[::1]:8000/api/v1; se borraron los resultados inválidos y se repitieron las 5 corridas	~5 min
2026-10-27	El primer inicio con Google devolvió 500 INTERNAL_ERROR (incidente JJA05WR4CG) y /health marcó la base «down»	Al editar api/.env para agregar GOOGLE_CLIENT_ID y GOOGLE_CLIENT_SECRET se borró un carácter de DB_PASSWORD: MySQL respondió «Access denied for user 'legrandhotel'»	Se restauró DB_PASSWORD igual a MYSQL_PASSWORD de docker-compose.yml; /health volvió a «ok». El código de incidente permitió hallar la causa en storage/logs en segundos	~1 min
2026-10-27	Después de entrar con Google el navegador mostró ERR_INVALID_REDIRECT	FRONTEND_URL lista dos dominios separados por coma (para CORS) y el callback de Google y la verificación de correo lo usaban completo como dirección de regreso	Nuevo App\Support\Frontend::url() que redirige al primer dominio; prueba que lo cubre en ClientPortalTest	~3 min
2026-10-27	k6 run se detenía en setup() con «La API no está sana antes de la prueba: 503»	El contenedor nginx de docker compose ocupaba el puerto 8000; k6 hablaba con ese Nginx (sin PHP detrás) y no con la API	docker compose down y encender solo MySQL (docker compose up -d mysql); el tutorial de k6 lo explica en el paso 2	~10 min
2026-10-27	Al volver de Google la página quedaba en blanco con #token=… en la dirección	La aplicación no llegó a arrancar en esa pestaña de Brave (la API nunca recibió /auth/me), probablemente por sus escudos (Shields); en Chrome el mismo flujo funcionó	Usar Chrome o desactivar Shields para localhost; se comprobó el flujo completo con una cuenta de prueba	~15 min

Fecha	Qué falló	Causa	Solución	Tiempo
2026-10-27	La pestaña «Errores» mostró la pantalla 500 con código WEB-MVRL1WBX	Laravel guarda algunas excepciones en el log como objeto (class, message, file) y la pantalla intentaba mostrarlas como texto (error de React)	RequestLogReader devuelve siempre la clase como texto; prueba nueva en DeveloperViewTest	~5 min

Cómo ayudó el código de incidente: cuando la API falla, responde 500 con un código (por ejemplo JJA05WR4CG) que el usuario ve en pantalla. El mismo código queda en el registro JSON de la API, y la Vista del programador lo encuentra en segundos (figura de la sección 4.4). Así se halló que el problema del primer inicio con Google era la contraseña de la base de datos, no Google.

Registro de la API (storage/logs/laravel.json), resumido

```
{"message": "SQLSTATE[HY000] [1045] Access denied for user 'legrandhotel'…",
 "context": {"incident_id": "JJA05WR4CG", "exception":
"Illuminate\\Database\\QueryException"},
 "level_name": "ERROR", "datetime": "2026-10-27T22:14:14-05:00"}
```

7. Pruebas de rendimiento y estrés con k6

k6 simula muchos «usuarios virtuales» (VU) usando la API a la vez y mide cuánto tarda cada respuesta. Cada guion tiene metas (thresholds): si no se cumplen, k6 marca la prueba con ✗ y termina con código 99. Las corridas se hicieron el 27 de octubre de 2026 en la laptop del equipo, con la API en el servidor integrado de PHP (php -S, que atiende una petición a la vez) y la base con 5000 reservas.

Prueba	Qué hace	Para qué
1. Humo (smoke)	5 VU durante 30 s piden /health y una ruta protegida sin token	Confirmar que la API, la base y la caché funcionan antes de cargar
2. Límite de login	6 intentos de login con contraseña incorrecta	Demostrar la protección contra fuerza bruta (429)
3 a 6. Escalones	El trabajo de recepción (70 % consultas, 20 % login, 10 % crear reserva) con 5, 10, 25 y 50 VU, 1 minuto cada uno	Ver desde cuándo la API se pone lenta y si aparecen errores al subir la carga

7.1 Prueba de humo

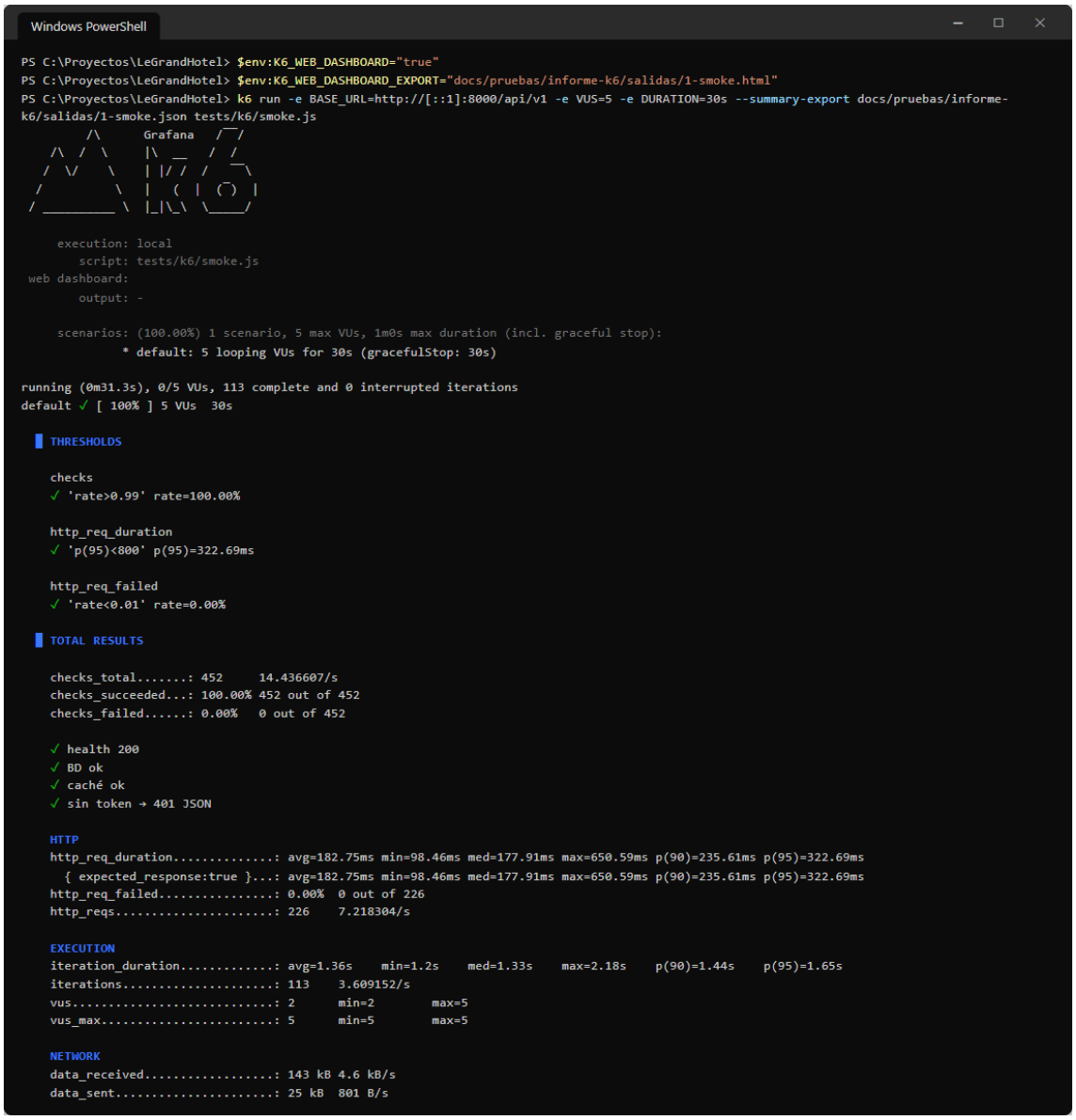


Figura 40. Comando de la prueba de humo y la respuesta de k6.

Interpretación: las tres metas salen con ✓. El 95 % de las respuestas tardó menos de 323 ms (meta: 800 ms), no hubo peticiones fallidas (0.00 %) y los 452 checks pasaron: la API, la base de datos y la caché responden, y una ruta protegida sin token devuelve 401 en JSON.

7.2 Límite de intentos de login

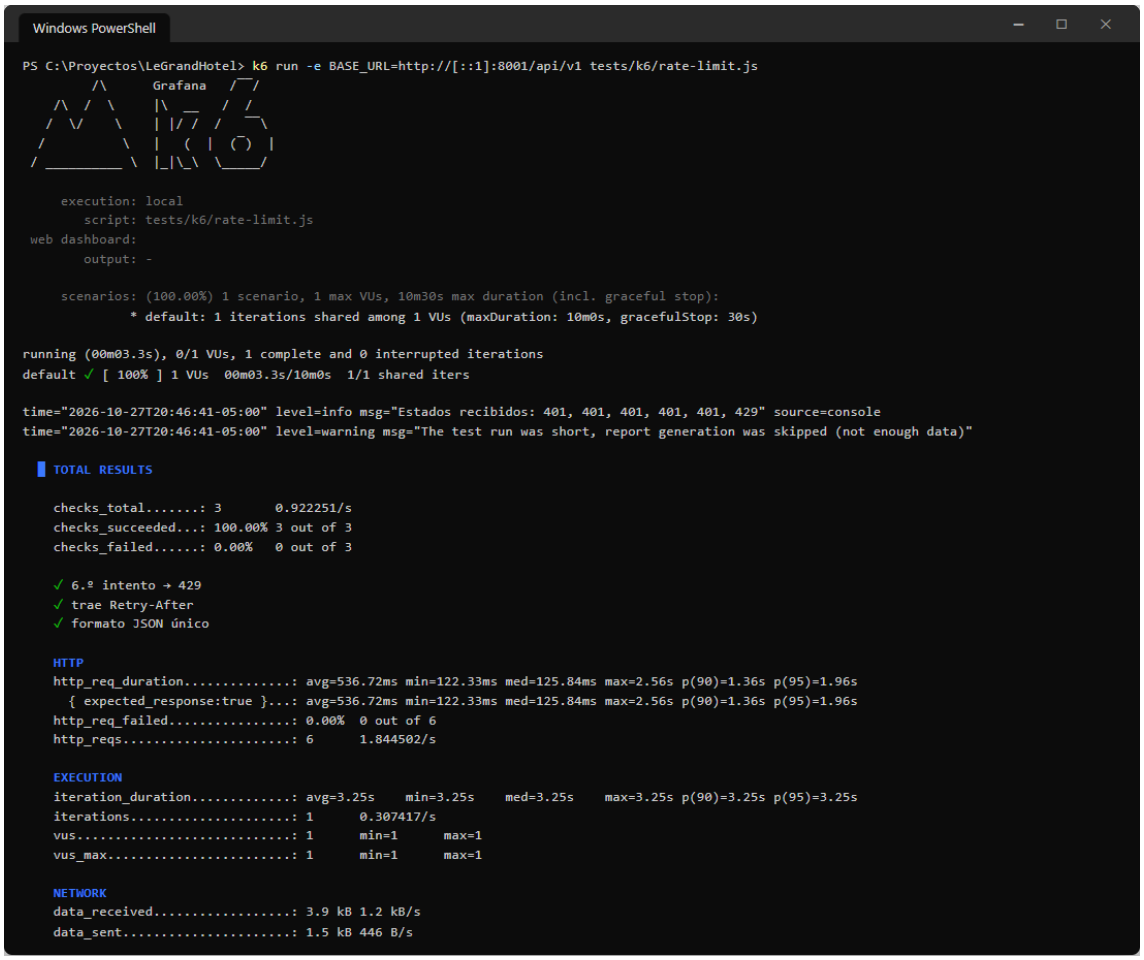


Figura 41. Prueba del límite de login: el sexto intento recibe 429.

Interpretación: «Estados recibidos: 401, 401, 401, 401, 401, 429». Los cinco primeros intentos responden 401 (contraseña incorrecta) y el sexto 429 con la cabecera Retry-After y el código TOO_MANY_REQUESTS. Nadie puede probar contraseñas sin límite.

7.3 Escalones de carga (5, 10, 25 y 50 usuarios)



Figura 42. Escalón de 5 usuarios: comando y respuesta.

Interpretación: con 5 usuarios todas las peticiones responden bien (0 % de errores y todos los checks ✓), pero la meta de velocidad sale con ✗: el p95 fue 610 ms y la meta es 500 ms. La línea final «thresholds … have been crossed» es k6 avisando que la prueba no se aprobó.



Figura 43. Escalón de 50 usuarios: comando y respuesta.

Interpretación: con 50 usuarios sigue sin haber errores (0.00 %), pero el p95 sube a 9,346 ms. La API no se cae: responde todo, solo que las peticiones esperan su turno.

Usuarios	Peticiones	Pet./s	Mediana	p95	Máximo	Errores	Meta p95 < 500 ms
5	308	4.8	304 ms	610 ms	1,131 ms	0.00 %	✗
10	320	4.9	1,155 ms	1,798 ms	2,037 ms	0.00 %	✗
25	367	4.8	3,936 ms	4,629 ms	5,010 ms	0.00 %	✗
50	452	4.8	7,922 ms	9,346 ms	9,823 ms	0.00 %	✗

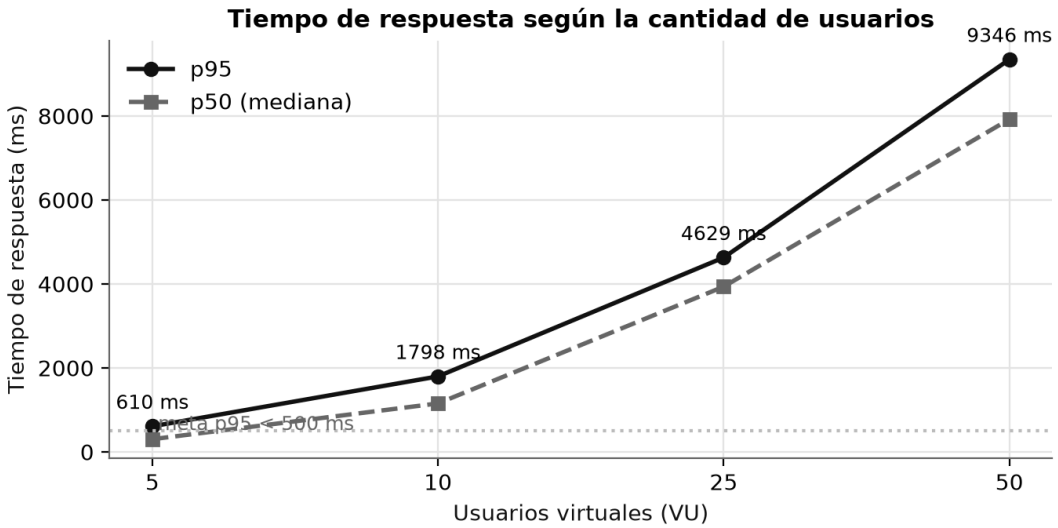


Figura 44. Tiempo de respuesta (mediana y p95) según la cantidad de usuarios.

Interpretación del gráfico: el eje horizontal son los usuarios a la vez y el vertical el tiempo de respuesta. La línea punteada es la meta de 500 ms. Las dos curvas suben casi en línea recta: al pasar de 5 a 50 usuarios el p95 crece de 610 ms a 9,346 ms. Que crezca en proporción a los usuarios es la señal de una cola: cada usuario nuevo espera a los anteriores.

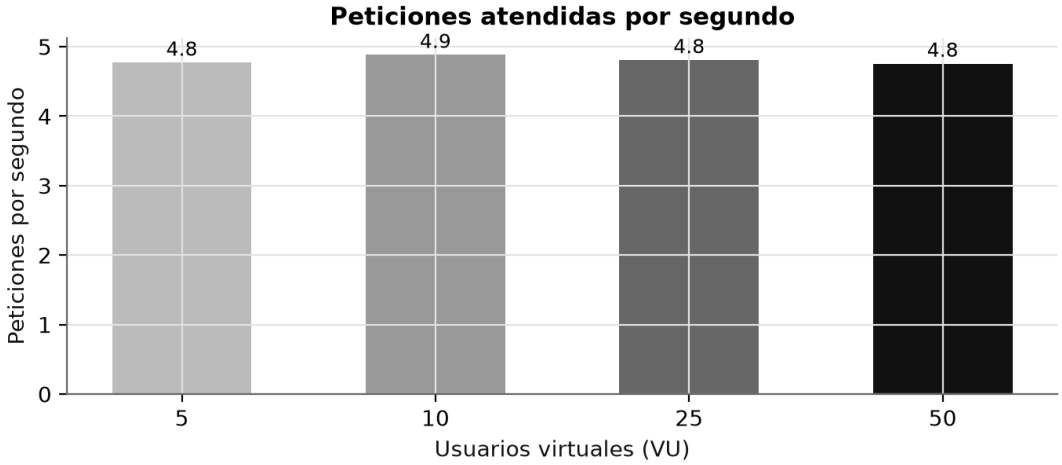


Figura 45. Peticiones atendidas por segundo en cada escalón.

Interpretación del gráfico: las barras casi no cambian (de 4.8 a 4.8 peticiones por segundo). Aunque haya diez veces más usuarios, la API atiende lo mismo por segundo: llegó a su techo. Junto con el gráfico anterior confirma el cuello de botella.

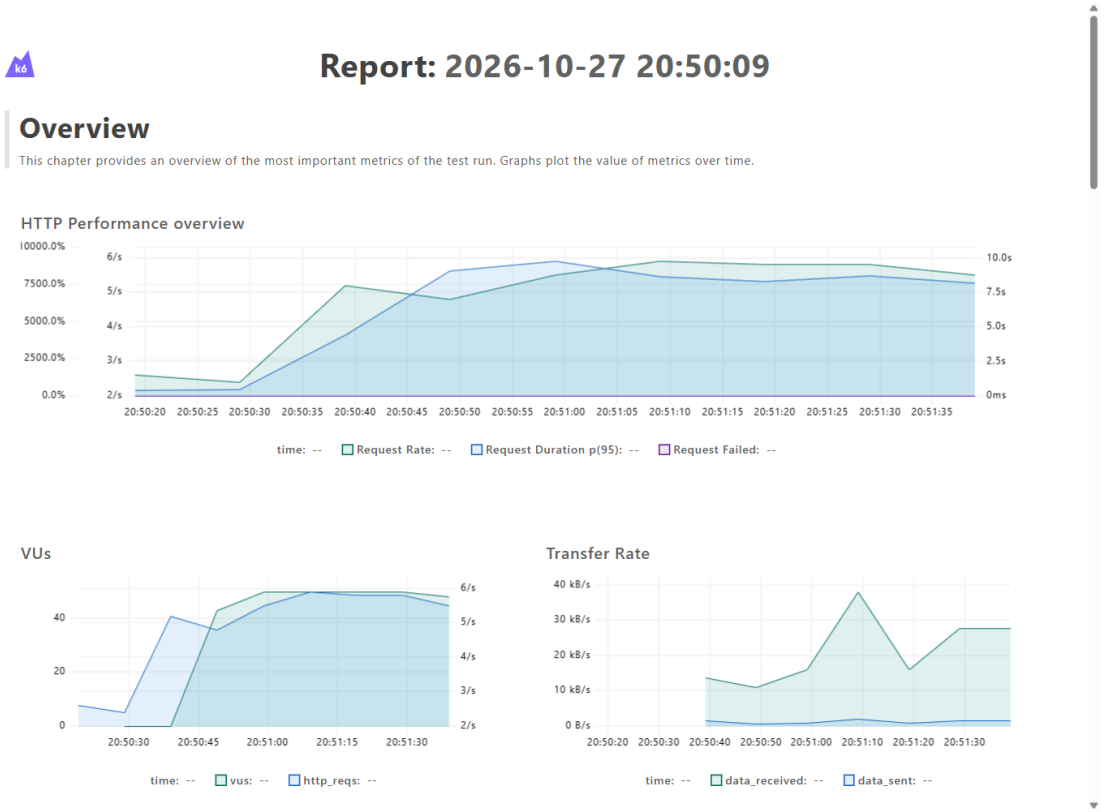


Figura 46. Reporte HTML de k6 del escalón de 50 usuarios.

Interpretación del reporte: en «HTTP Performance overview» la curva de peticiones por segundo se aplana en unas 5/s mientras el p95 sube hasta casi 10 s, y la de peticiones fallidas queda en 0. En «VUs» se ve la subida a 50 usuarios, la meseta de 45 segundos y la bajada.

7.4 Conclusión de las pruebas

- Correcta y estable: en las 6 corridas no hubo ni una petición fallida ni errores 500.
- Segura bajo carga: 401 sin sesión y 429 al pasar el límite de login, siempre en el formato JSON de la API.
- Lenta en la laptop por el servidor de desarrollo de PHP, que atiende una petición a la vez. La solución es correr la API con varios procesos PHP en paralelo y caché de código (OPcache) en el servidor definitivo, y repetir estas mismas pruebas allí, con más usuarios, cuando se defina el servidor.

8. Diseño de la aplicación Android e iOS

LeGrandHotel tendrá una aplicación para teléfonos que usa la misma API y los mismos roles que la web, sin duplicar ninguna regla del negocio. Cubre lo que conviene hacer desde el celular: el huésped reserva y revisa sus reservas, recepción atiende llegadas y salidas desde el lobby, y el almacén registra movimientos en el depósito.

Rol	Qué hará en la app
Cliente	Entrar con Google, completar DNI y teléfono, reservar por tipo de habitación, ver y cancelar sus reservas, recibir un recordatorio un día antes de llegar

Rol	Qué hará en la app
Recepcionista	Pantalla «Hoy» con llegadas y salidas, check-in y check-out, reservas, estado de habitaciones y cochera
Almacenero	Ver el inventario con los insumos bajo el mínimo y registrar entradas y salidas
Administrador	El panel resumido (ocupación, ingresos y alertas); la gestión completa sigue en la web

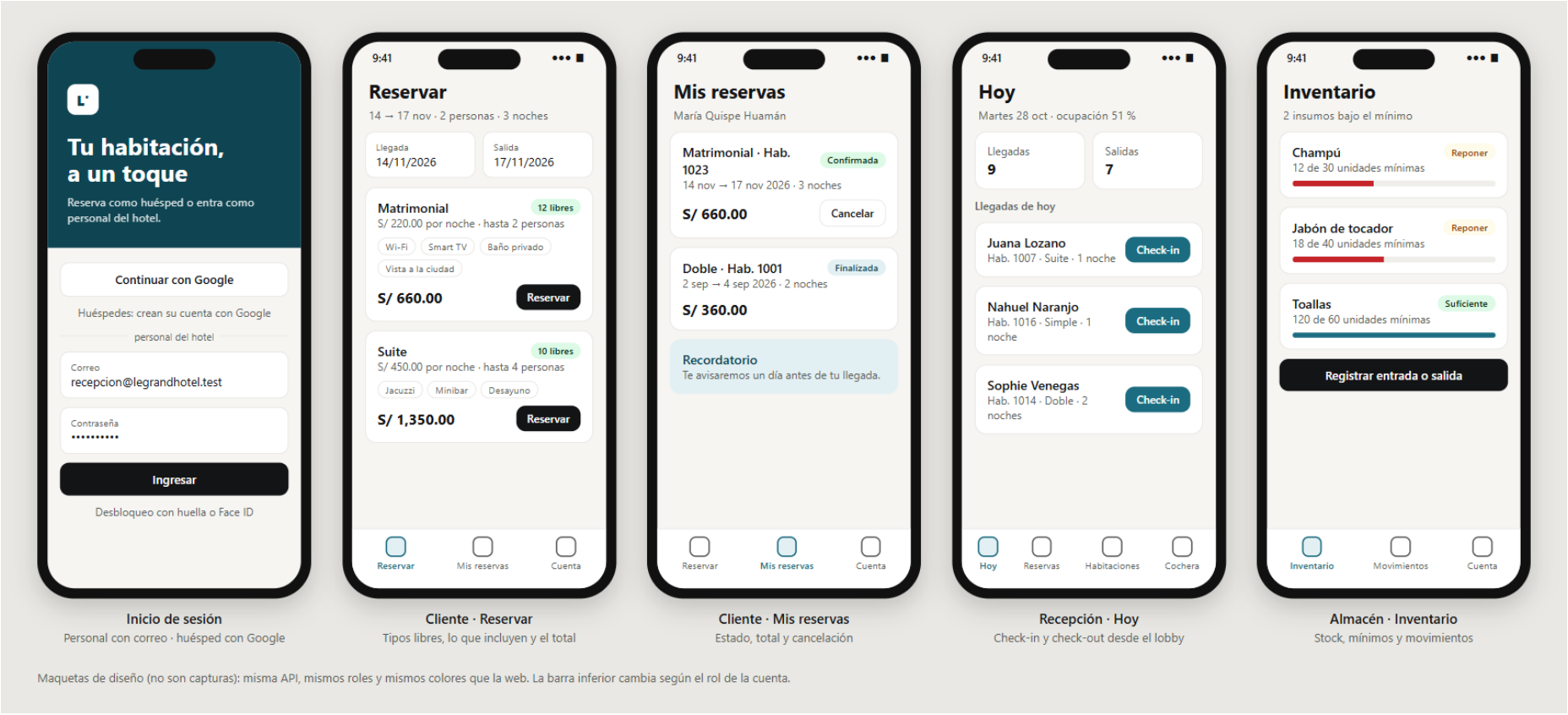


Figura 47. Maquetas de la app: inicio de sesión, Reservar y Mis reservas del cliente, «Hoy» de recepción e Inventario del almacén.

8.1 Tecnología

React Native con Expo y TypeScript. Con un solo código se generan las dos aplicaciones y se reutiliza lo que el equipo ya usa en la web: React, TypeScript, TanStack Query, Axios y los mismos tipos de datos.

Necesidad	En la app	Equivalente en la web
Navegación	Expo Router, con barra de pestañas según el rol	React Router
Llamadas a la API	TanStack Query y Axios con el mismo manejo de 401/403/429/5xx	Igual
Guardar la sesión	expo-secure-store: Keychain (iOS) y Keystore (Android)	sessionStorage
Inicio con Google	expo-auth-session con el enlace legrandhotel://auth/callback	Socialite con redirección
Notificaciones	expo-notifications (recordatorio de llegada, habitación lista, stock bajo)	—
Desbloqueo	Huella o Face ID (expo-local-authentication)	—
Pruebas	Jest y Testing Library; Maestro para recorridos completos	Vitest y Playwright

Opción evaluada	A favor	En contra	Decisión
React Native + Expo	Un código para Android e iOS; mismo lenguaje que la web; compila iOS sin Mac (EAS Build)	Funciones nativas muy específicas requieren módulos propios	Elegida
Flutter	Buen rendimiento y diseño uniforme	Otro lenguaje (Dart): no se reutiliza nada de la web	Descartada
Nativo (Kotlin + Swift)	Acceso total al sistema	Dos aplicaciones distintas: el doble de trabajo	Descartada

8.2 Seguridad y cambios en la API

- El token se guarda en el almacenamiento seguro del teléfono, nunca en texto plano, y se renueva con el mismo POST /auth/refresh de la web.
- La app no contiene secretos: las credenciales de Google y de la base de datos quedan solo en la API.
- La API casi no cambia: el login ya acepta un nombre de dispositivo (device_name) para que cada teléfono tenga su propio token. Faltará registrar el token de notificaciones de cada teléfono (POST /devices) y que el inicio con Google pueda volver a la app.

8.3 Plan de construcción

Etapa	Qué se construye
1	Proyecto Expo, login del personal y del cliente, sesión segura y barra de pestañas por rol
2	Cliente: completar registro, reservar y mis reservas
3	Recepción: «Hoy» con check-in y check-out, reservas y habitaciones; Almacén: inventario y movimientos
4	Notificaciones, desbloqueo con huella, pruebas y versión de prueba para Android e iOS

9. Conclusiones

- LeGrandHotel cubre la operación completa del hotel para cuatro roles del personal y la reserva en línea para los huéspedes, con 77 endpoints REST que responden siempre con el mismo formato.
- El registro y el login están separados: el personal entra con correo y contraseña y lo registran el Administrador o el Programador; los clientes entran solo con Google. Hay un solo Programador y la API lo hace cumplir.
- Los permisos se revisan dos veces: la web oculta lo que no corresponde y la API responde 403 aunque alguien escriba la dirección a mano.
- El registro de incidentes y el código de incidente permitieron encontrar y corregir cada falla el mismo día.
- Las pruebas k6 muestran una API correcta y sin errores bajo carga; su velocidad en la laptop está limitada por el servidor de desarrollo, y se medirá de nuevo cuando se defina el servidor.
- La app móvil reutilizará la API y las tecnologías de la web, y se construirá por etapas empezando por el login y el flujo del cliente.